Rakan

Viva Study Plan: from the idea to every line of code

Adaptive AI Fitness Coach with Real-Time Posture Correction

Item	Detail
Prepared for	Bos (Liang Qi Xun), BSc (Hons) Computer Science, INTI Penang / Coventry University
Prepared on	26 October 2026
Viva window	16 to 30 November 2026 (exact date not confirmed). About 60 minutes. Panel: supervisor and one examiner.
Study window	27 Oct to 15 Nov (20 days), then daily drills until the viva date. Plan assumes you could be called on 16 Nov.
Built from	Your repo (commit history to 972060c, 124 commits), HANDOVER.md, SYSTEM_OVERVIEW.md, docs/POSTURE_THRESHOLDS.md, the posture accuracy workbook, and the Chapter 5 emulator test log.
Format	13 modules. Each has: the story (idea and why), choices and rejected alternatives, files to read in order, worked examples with numbers, why it was tested that way, likely viva questions, and a teach-back task.

Read this first: what this document is and is not

• It is a **study plan and a reasoning guide**, not a script to memorise. Examiners can tell the difference within two questions.

• Facts marked **(doc)** come straight from your own files. Reasons marked **[C]** are reasons I inferred from your constraints. **Replace every [C] with your real reasoning** in your own words before the viva.

• I read the structure, the docs, the function lists and the study data. I did **not** read every line of all 30k lines of code. Each module therefore tells you exactly which function to open, so the code reading is done by you with a guide.

• Numbers (test counts, accuracy, timings) come from files dated 7 to 19 Oct. **Re-run the tests and re-check the numbers the week before the viva.**

Contents

Part	Section	What you get
A	1. How the viva works and how to study	The five-layer answer, the read-trace-break-teach method, daily time budget
A	2. Honest status snapshot	What is done, partly done, open; three tasks for this week
A	3. Day-by-day schedule, 27 Oct to 15 Nov	20 days, then daily drills
B	Module 0: The idea and the problem	Problem, objectives and how they changed
B	Module 1: Architecture and technology choices	Diagram, decision table, principles
B	Module 2: Flutter and Dart foundations	Boot, theme, navigation, vocabulary
B	Module 3: Authentication, rules, privacy	Throttle, verification, Firestore rules, defects
B	Module 4: Onboarding and data model	Which answer drives what; collection tree and why
B	Module 5: Plan generation	Rule-based generator, Phase 28 bug story
B	Module 6: Workout session and recovery model	Atomic save, MRV formula with example
B	Module 7: Posture detection	Pipeline, maths, analysers, thresholds, accuracy study
B	Module 8: Fatigue ML model	Features, formula, worked examples, honest R-squared
B	Module 9: Adaptive engine	Tiers, trend, missed days, plateau, decisions
B	Module 10: Coach, social, notifications	Lighter coverage
B	Module 11: Testing and evaluation	Why each test exists, defects found
B	Module 12: Limitations and viva game plan	Limitation answers, pitch, demo script
C	Question bank	46 questions with answer outlines
D	Appendices A, B, C	Numbers card, sources, file reading map

1. How the viva will actually work, and how to study for it

A 60-minute viva with a supervisor and an examiner is not a code review. It is a test of whether **you** made the decisions and can defend them. The examiner reads your dissertation, picks 4 to 6 places where a claim looks bold or an assumption looks unstated, and keeps asking "why" until you either show evidence or admit a limit. Your own notes say the supervisor has already flagged unstated assumptions and arbitrary thresholds, so expect exactly that.

The five-layer answer (use it for every topic)

Layer	Question it answers	Example for the squat depth threshold
1. Problem	Why does this exist?	Poor form causes injury (Chae et al., 2023), so the app must judge each rep.
2. Choice	What did you choose and what else was possible?	Rule-based angle check, not a trained classifier.
3. Evidence	What supports the number or method?	Parallel squat is about 60 to 70 degrees (Rojas-Jaramillo et al., 2024), plus a 10 degree tolerance gives 80.
4. Code	Where is it and how does it run?	`FormThresholds.squatDepthMax` in `angle_calculator.dart`, used by `SquatAnalyser` when `_RepTracker` closes a rep.
5. Test and limit	How do you know it works, and where does it fail?	Accuracy study: 91.3% on squats (150 reps). 2D only, exaggerated faults, small n.

The four-step method for reading code

- **Read** top-down: imports, class names, the public methods, then the private helpers. Do not read line by line yet.
- **Trace** one concrete input through it with a pencil (every module below has a worked example to copy).
- **Break** it on purpose: change one threshold or one constant, run the related test, and watch which test fails. A failing test teaches the boundary faster than reading.
- **Teach** it: explain the function aloud with the file closed. If you get stuck, that is the exact spot you will get stuck in the viva. Record yourself on your phone.

Time budget per study day (about 2.5 hours)

Block	Time	What
Story and reasons	30 min	Read the module's story and alternatives. Rewrite every [C] reason in your own words.
Code	75 min	Open the listed files in order. Trace the worked example. Run the related tests.
Teach-back	30 min	Do the teach-back task aloud, no notes. Record it.
Question drill	15 min	Answer the module's viva questions aloud. Mark the ones you stumbled on for the next day's warm-up.

Evidence labels used in this document

• **(doc)** = taken from your HANDOVER, SYSTEM_OVERVIEW, POSTURE_THRESHOLDS, study workbook or test log.

• **[C]** = reason inferred from your constraints (zero cost, solo developer, free tools, run locally). Confirm or replace it.

• **CHECK** = something I could not verify from the files. Resolve it before the viva.

2. Honest status snapshot (know this before anyone asks)

Examiners reward a student who states what is done, what is partly done, and what is not done. They punish discovered gaps.

Area	Status	Evidence	What to say
Adaptive planner built	Done	Plan generator, 7 adaptive mechanisms, missed-day flow, plateau detection	Built and device-tested on Samsung S20 FE (Scenarios A to E).
Posture detection built	Done: 31 exercises, 7 analyser classes	Phase 29; `FormThresholds` with sources	Rule-based, 2D, on-device.
Posture accuracy study (Objective 2)	Done for squat and push-up only	5 testers x 3 sets x 10 reps each; squat 91.3%, push-up 100%, overall 95.7%	Controlled study; deadlift has no data (no spine landmarks).
Study data integrity	Explained, must be framed carefully	T02 squat set 2: 4 cells edited after the session because they could not be corrected live. The edited column is `predicted_correct` (the app's verdict), not the ground truth.	Headline uses the verdict recomputed from the logged angle, so the edits do not affect 91.3%. Disclose them; see Module 7e.
UAT (>=75% satisfaction, Objective 1)	Done (22 to 23 Oct)	Google Form responses: 20 participants (P01 to P20), consent recorded. SUS mean 85.1 (SD 6.6, range 72.5 to 95). Overall satisfaction (F8) 19/20 = 95% agree.	Target met. Cross-check these figures against the UAT section of your FYP report (I could not open the report itself).
Fatigue-model effectiveness (Objective 3)	CHECK	HANDOVER says built, effectiveness not evaluated	Be ready to say what you built, what you evaluated (unit boundaries, simulated weeks), and what a real evaluation would need.
Plateau detection on device	Partly done	Unit-tested; Chapter 5 log shows device pass (CCH-06 with seeded sessions)	Device-verified with seeded data; the hidden-below-5-sessions part is covered by unit tests.
Emulator system testing	Done 18 Oct	RESULTS_LOG: test IDs AUTH, ONB, HOM, WRK, SCH, REC, CCH, SOC, SET, LIB, IT, ST; several FAILs found (see Module 11)	Know the FAILs and whether each is fixed (check `git log`).
Backend endpoints unauthenticated	Known limitation	ST-14: `/generate-plan` with another uid and no token returned HTTP 200	Honest limitation; fix = verify Firebase ID token server-side.
ML training data	Synthetic	Labels from a hand-written formula	R-squared 0.83 shows the model recovers the formula, not real-world validity.

Three things to do this week, before any studying

• Learn the exact wording for the 4 edited squat cells and the 5 observer flips (Module 7e). This is still the most likely place for a hard question.

• Run `flutter test`, `flutter analyze` and `pytest` (backend). Write the real numbers on a card. Older documents say 53 tests (Phase 27); the Oct 18 log says 105 passed.

• Check that the UAT figures in your report match this plan (SUS 85.1, 95% overall satisfaction, 20 participants). If they differ, the report is the source of truth; tell me and I will update the plan.

3. Day-by-day schedule: 27 Oct to 15 Nov

Weeks 1 and 2 build understanding from the ground up. Week 3 is for joining it up, mocking the viva, and rehearsing the demo. Every 4th day is a lighter teach-back day.

Day	Date	Module	Goal for the day
1	Tue 27 Oct	Module 0	The idea, the problem statement, how objectives evolved. Write your 3-minute pitch (first draft).
2	Wed 28 Oct	Module 1	Architecture and technology choices. Draw the architecture diagram from memory.
3	Thu 29 Oct	Module 2	Flutter/Dart foundations through your own boot, theme and navigation code.
4	Fri 30 Oct	Module 3	Auth, email verification, login throttle, Firestore security rules.
5	Sat 31 Oct	Module 4	Onboarding and the Firestore data model. Redraw the collection tree.
6	Sun 1 Nov	Module 5	Backend plan generator (Python). Run its tests.
7	Mon 2 Nov	Review 1	Teach modules 0 to 5 aloud in 15 minutes. Answer the 15 weakest questions. Re-read your UAT results and method.
8	Tue 3 Nov	Module 6	Workout session, logging, atomic save, muscle recovery model.
9	Wed 4 Nov	Module 7a	Posture pipeline: Kotlin, CameraX, MediaPipe, platform channels.
10	Thu 5 Nov	Module 7b	Angle maths, `_RepTracker`, the analysers. Hand-calculate three angles.
11	Fri 6 Nov	Module 7c	Thresholds, literature, Phase 29 corrections, excluded exercises.
12	Sat 7 Nov	Module 7d	Accuracy study: design, metrics, results, data issues. Rebuild the confusion matrix by hand.
13	Sun 8 Nov	Module 8	Fatigue ML model: data generation, training, coefficients, thresholds.
14	Mon 9 Nov	Review 2	Teach modules 6 to 8 in 15 minutes. Mock Q&A on posture only (20 min).
15	Tue 10 Nov	Module 9a	Adaptive engine: propose/commit, tiers, trend, weekly summary service.
16	Wed 11 Nov	Module 9b	Missed-day flow, reschedule rules, plateau detection, decisions #44 to #60.
17	Thu 12 Nov	Modules 10 and 11	Coach, social, notifications (light). Testing: why each test exists, defects found.
18	Fri 13 Nov	Module 12	Limitations, future work, honest answers. Citation table.
19	Sat 14 Nov	Mock viva	Full 60-minute mock with a friend or recorded alone. Use the question bank in Part C at the end.
20	Sun 15 Nov	Demo and card	Demo rehearsal on the real phone, Render warm-up, one-page numbers card, early night.

After 15 Nov (until your viva date is confirmed)

- 30 minutes a day: 10 random questions from the bank, aloud. Re-read Appendix A (numbers).
- Two days before the viva: run the demo twice on the phone, charge it, load a screen recording as a backup (Render may be asleep).
- The day before: no new material. Read only the section 2 snapshot, the limitation table, and your pitch.

If you fall behind

- Protect Modules 7 (posture), 8 (ML), 9 (adaptive) and 11 (testing). They carry the marks and the hardest questions.
- Compress Modules 2, 10 and 6 to a one-hour skim each. Never skip the limitations module.

MODULE 0 The idea and the problem

Why Rakan exists, who it is for, and how the objectives evolved. Everything else in the viva hangs off this story.

The story in four sentences

Millions of people start exercising and stop. The proposal cites that about **31.3% of adults (1.8 billion) were insufficiently active in 2022** (Strain et al., 2024), that **around 50% drop out within six months** (Linke et al., 2011), and that **poor form is a major cause of training injuries**, with an AI coaching app improving posture correction in a randomised trial (Chae et al., 2023). Generic apps hand out a fixed plan and never look at how you lift or how tired you are. Rakan is a **coach** (a plan that adapts to your logged effort and recovery), a **spotter** (the phone camera checks each rep's form) and a **physio-aware planner** (it routes around injuries), built with free tools only.

How the objectives changed (you must own this table)

	Original proposal	Current (HANDOVER)	Why it changed (outline your answer)
Obj 1	Adaptive 7-day planner, >=80% user satisfaction	Same, target **75%** via user acceptance testing, plus performance dashboard	A defensible target for a small cohort. **Result: 95% overall satisfaction, SUS 85.1 (n = 20).**
Obj 2	MediaPipe posture feedback, **>=90%** accuracy on squat, push-up, deadlift	Same exercises, target **75%**	90% was set before the limits of single-camera 2D pose were known. Dill et al. (2023) report accuracy degrading with conditions; your tolerance design (10 degrees) comes from this. 75% is a stated, citable minimum. Your result (95.7% overall) exceeds it, but under controlled conditions.
Obj 3	Evaluate effectiveness over 4 weeks (form and adherence)	Fatigue and recovery monitoring from RPE, evaluate effect on overtraining risk	Narrowed to what a single semester can measure and what the code actually does.

Features: proposed vs delivered

Proposed feature	In the code today
Adaptive workout planner (rule-based plus scikit-learn)	Rule-based plan generator (Python) plus a linear-regression fatigue model; weekly propose/commit adaptation with 5 priority tiers.
Real-time posture correction (squat, push-up, deadlift)	31 exercises across squat, lunge, deadlift, Romanian deadlift, push-up family, shoulder press, curl family; voice and HUD feedback; rep counting.
Performance and fatigue analysis (RPE, soreness)	RPE 1 to 10 per exercise; per-muscle recovery score from weekly volume vs MRV; injury logger and recovery map.
Recovery tracking (sleep optional)	Muscle recovery and injuries built. **CHECK:** I found no sleep tracking in the code survey; treat it as future work.
Progress dashboard	Coach tab: volume, consistency, muscle focus, per-exercise progression, PR records, weight log, plateau notice.
Exercise library with pose overlay	626-exercise library with GIF demos; skeleton overlay in the posture screen.
(not proposed) Social layer, local reminders, account deletion	Added to support adherence (the dropout problem) and Google Play requirements.

Alternatives and gaps you should be able to name [C]

- **Existing apps** (generic plan apps, video-based form coaches) usually do one of: static plans, or form feedback without plan adaptation, often behind subscriptions. Rakan combines both at zero running cost. Be ready to name two real apps and one concrete difference; do not guess here, look them up.
- **Why mobile, not wearable or web:** the phone camera is the cheapest sensor everyone owns; the gym is where the user is.

- **Why Malaysian users:** local target audience; equipment profiles (home, few equipment, full gym) match onboarding step 5.

Viva questions for Module 0

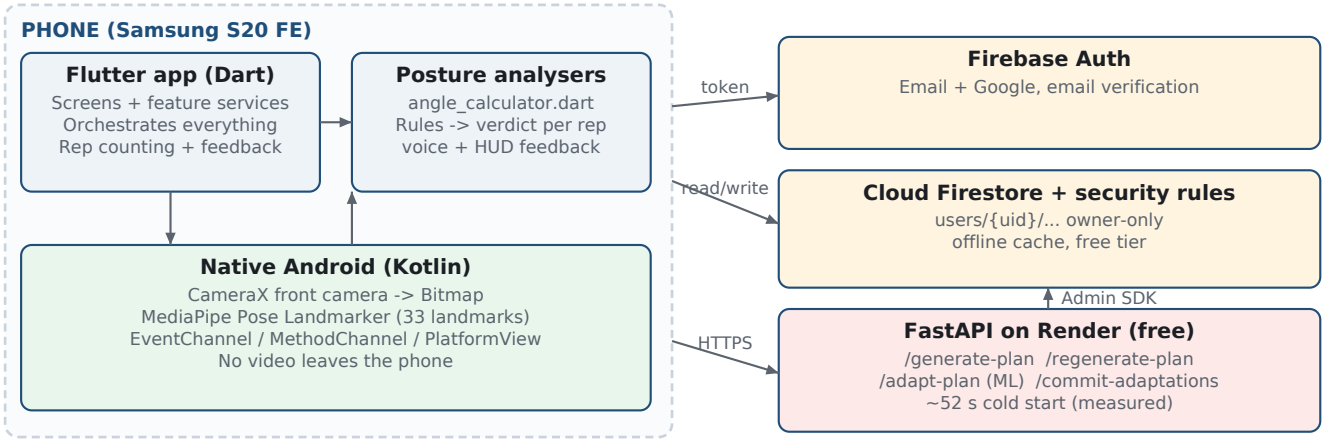
- Why did you choose this problem? What is the gap in existing solutions?
- Why did the accuracy target drop from 90% to 75%? Is that lowering the bar?
- Which objective is best evidenced, and which is weakest?
- Who is the user and what does Rakan do for a complete beginner versus an advanced lifter?
- What would you do with another six months?

Teach-back task

• Deliver the 3-minute pitch aloud: problem (two statistics), solution (coach, spotter, physio-aware), how it is built (free stack), what you proved (accuracy study), what you did not prove (UAT, effectiveness). Record it and listen once.

MODULE 1 Architecture and technology choices

Every box in the diagram is a decision someone will ask you to defend. The common thread is your own project constraints: free tools, zero cost, runs locally, solo developer.



Reading the diagram. The phone does the heavy, private work (camera, pose estimation, rep judgement, most reads and writes). Firebase provides identity and storage. The Python backend is deliberately small: it generates and regenerates plans (the only place the Admin SDK writes to Firestore), and it exposes two stateless calculators (`/adapt-plan`, `/commit-adaptations`) that take numbers in and return numbers out (doc). **Orchestration lives in the Flutter client**: `WeeklySummaryService` gathers data, calls the backend, then applies the result in one Firestore batch.

Decision table: chosen, alternatives, reason, trade-off

Decision	Chosen	Alternatives	Why	Trade-off you admit
Mobile framework	Flutter (Dart)	Native Kotlin/Swift; React Native	One codebase for Android and iOS; fast UI iteration; named in the proposal (doc). [C]	Pose detection is Android-only because MediaPipe runs natively in Kotlin.
Pose estimation	MediaPipe Pose Landmarker, 33 landmarks	OpenPose, MoveNet, ML Kit pose	Free, on-device, real-time on a phone, in the proposal (doc). Runs without sending video anywhere (privacy, no server cost).	2D only; no spine points; accuracy depends on camera angle.
Backend	FastAPI on Render free tier	Firebase Cloud Functions; Flask; Django	Python is needed for scikit-learn; Pydantic validates inputs; Cloud Functions needs a paid plan. Zero cost constraint (doc). [C]	~52 s cold start (measured 52.37 s); unauthenticated endpoints.
Database	Cloud Firestore	PostgreSQL (the other option in the proposal)	No server to host; offline cache (proved in test IT-10); security rules enforce ownership; free tier. [C]	No joins; data is shaped around queries; large-collection reads are costly.
Auth	Firebase Auth (email and Google)	Custom JWT/auth server	Free, handles verification emails and password reset, integrates with rules.	Client-side login throttle is only friction; the server also rate-limits.
ML	scikit-learn LinearRegression + StandardScaler	Random forest, neural net	Interpretable coefficients, tiny feature set, project rule: fundamentals before advanced AI (doc, project instructions).	Trained on synthetic labels.
Images	Base64 inside Firestore (384x384, quality 70)	Firebase Storage	Storage needs the paid Blaze plan, confirmed in Phase 23 (doc).	Document size limit; must compress; extra read cost.
Reminders	Local notifications	Firebase Cloud Messaging	Works offline, no backend, no cost (doc).	Reminder settings are per device, not per account (seen in testing).
Posture logic	Rule-based angle thresholds	Train a classifier on Kaggle landmark datasets	Explainable per-joint feedback, no labelled-fault data needed, threshold values traceable to literature, real-time. [C]	Thresholds are only as good as the sources; estimates are labelled.

Two architectural principles that explain many choices (doc)

- **Stateless rules API.** The backend functions take numbers and return numbers. That makes them cheap to host, trivial to unit-test with pytest, and keeps Firestore access in one place. Decision #58 repeats this: plateau detection stayed in Flutter because adding Firestore reads to the stateless backend was disproportionate.
- **Propose, then commit.** A session only writes a pending proposal; a weekly job commits adjustments. This stops the plan from lurching after one bad day (Module 9).
- **Fail-closed defaults.** Wrong or missing data defaults to exclusion or a safe state (equipment matching, day eligibility). This is a named standing rule in your engineering principles.
- **Polyglot drift** is a named risk: Dart and Python keep separate vocabularies ("abs" vs "Core"), so normalisation happens at the boundary, and the same fix was applied in both languages for `noEquipment` exercises.

Viva questions for Module 1

- Why not put everything in the app and drop the backend? (Answer: scikit-learn and rule code in Python; plan write via Admin SDK; the cost is cold start. Be honest that a pure on-device port is possible future work.)
- Why Firestore and not PostgreSQL? What do you lose?
- What happens when the Render server is asleep? (Plan generation shows a loading state with a 90 s timeout; adaptation call failure is swallowed so the session still saves.)

- Where does user data go? Does any video leave the phone? (No.)
- What would break first if 10,000 users joined? (Cold start, unauthenticated endpoints, full-collection reads, Render free-tier limits.)

Teach-back task

• Redraw the architecture diagram on paper in under 2 minutes, then narrate one full journey: user finishes a workout -> data saved -> `/adapt-plan` called -> proposal written -> Sunday commit -> plan changes dialog.

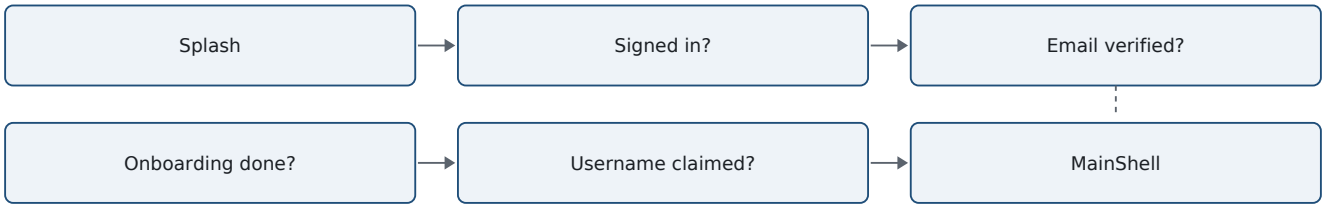
MODULE 2 App skeleton: Flutter and Dart foundations

If the examiner opens a Dart file and points at a line, you need the vocabulary. Learn Flutter through your own boot, theme and navigation code, which is short.

Read in this order

File (lines)	What to look for
`lib/main.dart` (16), `lib/app.dart` (16)	`WidgetsFlutterBinding`, `Firebase.initializeApp`, `runApp`; `MaterialApp` with the dark theme; the first screen is the splash.
`lib/firebase_options.dart` (88)	Generated config for project `rakan-10680`. You did not write it; know what it is.
`core/theme/app_colors.dart` (28), `app_typography.dart` (49), `app_theme.dart` (121)	Design tokens: background `#0C0E10`, primary `#C6C6C7`, error `#EE7D77`; Space Grotesk for headlines and Manrope for body; no borders, tonal depth.
`shared/widgets/main_shell.dart` (69), `bottom_nav_bar.dart` (151)	`IndexedStack` of four tabs (Home, Workout, Coach, Settings) keeps each tab's state alive.
`features/onboarding/screens/splash_screen.dart` (536)	Animated brand, then calls `AuthNavigationService.resolveNextScreen()`; retry screen if routing fails (Phase 28).
`features/auth/services/auth_navigation_service.dart` (40)	The routing decision tree below.

Startup routing (draw this)



No user -> Login. Signed in but unverified -> Email verification screen. Verified but no profile flag `onboardingCompleted` -> Onboarding. No username (legacy accounts) -> Choose username. Otherwise -> `MainShell`. (doc: SYSTEM_OVERVIEW 6.1.)

Dart/Flutter vocabulary you must be able to use

Concept	One-line meaning	Where in Rakan
Widget tree	UI is a tree of small objects rebuilt when state changes.	Every screen file.
StatelessWidget vs StatefulWidget	Stateful keeps mutable `State`; `setState` triggers rebuild.	`workout_active_screen.dart` (set values, timers).
Future / async / await	A value that arrives later (network, Firestore).	Every `Future<...>` service method.
Stream	A sequence of values over time.	Pose landmarks: `PoseService.getLandmarkStream`.
BuildContext and Navigator	Position in the tree; stack-based screen navigation.	`Navigator.push`, plan generation clears the stack.
Platform channels	Dart-to-native bridge: `MethodChannel` (call and reply), `EventChannel` (stream), platform view (native view embedded in Flutter).	`com.example.rakan/pose_landmarks`, `/permissions`, `/camera_preview`.
Static pure functions	No side effects, easy to test.	`AdaptService.computeMissedDays`, `detectPlateau`.
Null safety	`String?` can be null; `!` asserts not null; `??` gives a default.	`profile['experienceLevel'] ?? 'beginner'`.

Design system reasoning [C]

- A dark, low-glare palette suits gyms and phone cameras in dim light and looks premium. "No borders, tonal depth" keeps layouts uncluttered.
- A single set of tokens (`AppColors`, `AppTypography`) means one change updates the whole app; this is the same *one source of truth* rule you apply to code.
- Standing style rule: `withValues(alpha: x)`, not the deprecated `withOpacity()`. Phase 28 converted all 69 occurrences.

Viva questions for Module 2

- Why `IndexedStack` instead of rebuilding tabs?
- What is the difference between a `MethodChannel` and an `EventChannel`, and which does pose detection use for the landmark stream?
- How does the app decide which screen to show at launch?
- Why a design system document in an FYP? (Consistency, one source of truth, maintainability.)

Teach-back task

• Open `main.dart` and walk from `main()` to the first pixel on screen, naming every class on the way. Then describe what changes if `Firebase.initializeApp` is removed.

MODULE 3 Authentication, security rules and privacy

Security questions are easy marks if you know four mechanisms: email verification, login throttle, Firestore security rules, and the honest backend gap.

Read in this order

File (lines)	What to look for
`auth_service.dart` (182)	Sign-in, registration, Google sign-in, password reset; `reloadAndCheckVerified` (see defect D-28 below).
`login_throttle.dart` (52)	Persisted in `SharedPreferences`. 5 free failures, then a 30 s lockout that doubles up to a 15-minute cap (doc). Survives app restart (test AUTH-06).
`login_screen.dart` (501), `register_screen.dart` (575)	Generic error text ("Incorrect email or password") so an attacker cannot learn which emails exist; password-strength checklist (8+ characters, lower, upper, digit, symbol).
`email_verification_screen.dart` (244)	Polls every 5 s; resend cooldown 60 s.
`firestore.rules` (129)	`isVerified()`, owner-only rules, `canViewActivity()`, follow rules.
`account_deletion_service.dart` (178)	Re-authenticate, delete data, follows, username and the Auth user (Google Play requirement).

How Firestore security rules work (explain in 60 seconds)

The client talks to Firestore directly, so the **rules file is the real access control**. Each read/write is checked on Google's servers against rules that can inspect the signed-in user (`request.auth.uid`, `email_verified`), the request data and other documents. In Rakan: (1) everything under `users/{uid}/**` is owner-only and requires a verified email; (2) the top-level `users/{uid}` index doc (display name, username, photo, `isPrivate`) is readable by verified users, writable only by the owner; (3) `usernames/*` has create/delete but no update, to keep names unique; (4) `activityFeed` (a thin, weight-free copy of each session) is visible to the owner, anyone if the account is public, or an accepted follower if private; (5) `follows/{followerUid_followingUid}` can only be created by the follower with a matching doc id, and `accepted` is allowed at creation only if the target is public (Phase 28 fix); (6) comments are 1 to 500 characters.

Why the data is structured for privacy [C]

- Followers never get read access to `workoutLogs`. They read `activityFeed`, a **denormalised copy** that deliberately omits weights and volume. Privacy by structure, not only by rule.
- Deterministic follow doc ids make "do I follow X?" a single read and let a rule verify the id.
- Follower counts use `count()` aggregation instead of counters, because a counter would need cross-user writes that owner-only rules forbid (no Cloud Functions on the free plan).

Bugs the tests found here (know them, they show your testing worked)

ID	Finding (from Chapter 5 log)	Lesson
D-24 (High)	Users whose `users/{uid}` doc lacked an `isPrivate` field could not be followed. Rules read a missing field as an error and deny; the client treated missing as public.	Client and rules must agree on defaults. Test with accounts that have never touched a setting. (Check whether you fixed this: `git log` and `PublicProfileService`.)
D-28 (Medium)	Offline relaunch showed "Verify your email" because a network error during `reload()` was treated as "not verified".	Distinguish network failure from a negative result. (Fail-closed is not the same as fail-wrong.)
D-23 / AUTH-07, AUTH-14	Success snackbars were invisible: text had no colour on a dark surface.	Pixel check proved the text colour equalled the background.
ST-14	`/generate-plan` accepted another user's uid with no token and wrote a plan (HTTP 200).	Admin SDK bypasses rules, so the backend must authenticate. This is a documented limitation.

Viva questions for Module 3

- How do you stop user A reading user B's workouts? Show me the rule.
- Is the backend secure? (No: be precise. It validates input ranges, but it trusts the uid in the request. Fix: verify the Firebase ID token with the Admin SDK on each call.)
- Why does a login throttle exist if Firebase already limits attempts?
- Why does the feed store a copy instead of letting followers read logs?
- Where do you store the service-account key and how do you keep it out of GitHub? (`backend/.gitignore` lists `serviceAccountKey.json`; it is not tracked; on Render it comes from the `FIREBASE_CREDENTIALS_JSON` environment variable.)

Teach-back task

• Take the rule for `activityFeed` reads and explain, with three example users (owner, stranger, accepted follower) and one private account, which reads succeed and why.

MODULE 4 Onboarding and the Firestore data model

Why eight questions, what each one feeds, and why the database is shaped the way it is. Data-modelling logic is an explicit requirement of your project brief.

Read in this order

File (lines)	What to look for
`onboarding_data.dart` (319)	One mutable object carrying all answers; per-step validity getters; `toMap()` writes the profile; metric-first storage (`heightCm`, `weightKg`) with an `isMetric` display flag.
`onboarding_shell.dart` (249)	Step host with a progress bar; one or two questions at a time (from the app design brief).
`steps/step1..step8` (991 / 241 / 277 / 360 / 116 / 219 / 362 / 634)	Profile, goal, experience, preference (days and duration), environment (equipment), motivation, focus areas, safety (injuries via body-map painter).
`body_map_painter.dart` (433)	Custom painter for the tappable body map (16 regions).
`user_profile_service.dart` (133), `plan_service.dart` (45)	Save profile; call `/generate-plan`.
`plan_generation_screen.dart` (397)	Loading screen with a 90 s timeout for the Render cold start.

Which answer drives which part of the system

Answer	Used by	Effect
Goal (muscle gain, weight loss, endurance, flexibility)	`GOAL_PRESCRIPTION` in `plan_generator.py`	Shifts reps and rest (endurance 15 to 20 reps and rest of 45 s or less; weight loss and flexibility 12 to 15 reps and 60 s or less). **Does not change which exercises are picked.**
Experience	Exercise count bounds; difficulty filter; ML feature	Beginner 3 to 6, intermediate 3 to 7, advanced 3 to 8 exercises per session.
Equipment	Exercise pool filter	Fail-closed; `noEquipment` exercises bypass the check.
Days per week and duration	Split template; session fill	1 to 7 days map to full body, upper/lower, PPL and so on; sessions fill the chosen minutes less 10 for warm-up and cool-down.
Focus areas	Selection tier 2	Extra exercises only if they fit that day's split.
Injuries	`/generate-plan` and `/regenerate-plan`	Injured muscle groups are excluded; days can become "Recovery (injury-adjusted)".
Age, gender, height, weight, activity	Profile, stats, BMI display	Not used in plan generation (be honest if asked).

Data modelling logic (the tree, and why it is shaped this way)

```
users/{uid}                       public index doc (name, username, photo, isPrivate)
  profile/data                    private questionnaire answers
  workoutPlans/{planId}           plan header (status, weekNumber)
    days/{dayPlanId}              7 day docs (dayType: workout | rest)
      exercises/{id}              sets, reps, rest, order, adaptedAt
  workoutLogs/{logId}             one finished session
    exerciseLogs/{id}             per-exercise sets + rpeScale
  activityFeed/{logId}            thin follower-visible copy (no weights)
  adaptationProposals/{id}        pending / committed
  muscleRecovery/{group}          fatigueScore, weekly MRV
  weeklySummaries, scheduleOverrides, missedDayResolutions,
  weightRecords, injuries, progressPhotos
usernames/{name} -> {uid}         uniqueness index
follows/{follower_following}      {status: pending | accepted}
```

- **Query-first design.** Firestore has no joins, so the shape follows what the screens ask for: today's plan day, recent logs, a user's recovery map. Nesting under `users/{uid}` also makes ownership a one-line rule.
- **plan -> days -> exercises** lets the adaptation engine update only future days' exercises with one batch write.
- **Dates as ISO strings** avoid composite indexes; Phase 28 then added `orderBy('completedAt').limit(n)` (single-field, automatic index) so reads no longer scan whole collections.
- **Usernames in a transaction** guarantees uniqueness without a server.
- **Progress photos moved** from the log document to `progressPhotos/{logId}` (Phase 28) to keep logs small; a one-time background migration moves old photos.
- **Gotcha:** `exerciseName` is the only reliable key to look up exercise metadata; `exerciseId` is a backend UUID unrelated to the static library (doc).

Viva questions for Module 4

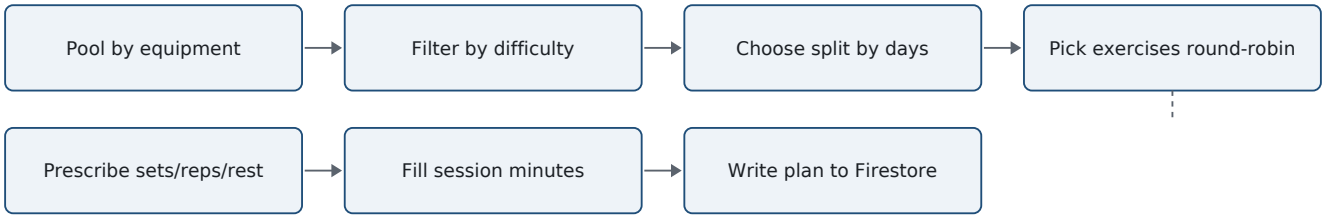
- Why eight steps? What would you drop? (Show which answer drives which feature; unused fields are future work.)
- Why store height and weight metric-first?
- Why a separate `activityFeed` collection?
- Firestore is NoSQL. How did you decide what to nest and what to denormalise?
- What happens to old data when the plan is reset? (Old plan is retired in the same batch that publishes the new one, Phase 28 fix; logs are untouched.)

Teach-back task

• Pick the exercise "Push-Up" and describe every Firestore document that exists after one completed session, with field names. Then explain which of them a follower can read.

MODULE 5 Plan generation: the rule-based backend

The "AI planner" in its simplest honest form: deterministic rules, tiered selection, and a history of self-found bugs that make a good viva story.



Read in this order (`backend/services/plan_generator.py`, 565 lines)

Symbol	What it does
`EXERCISE_COUNT_BOUNDS`, `SESSION_MINUTES`, `WARMUP_COOLDOWN_MINUTES = 10`, `SECONDS_PER_WORKING_SET = 45`, `MAX_SETS_PER_EXERCISE = 5`	The constants behind session sizing. Know why 45 s per set and 10 minutes of warm-up. [C]
`DAY_SPLITS`, `SPLIT_MUSCLES`	1 day full body; 2 upper/lower; 3 push/pull/legs; 4 PPL plus full; 5 and 6 body-part splits; 7 adds active recovery.
`_target_group(ex)`	Resolves "arms" into biceps or triceps using keywords so pull days stop getting tricep dips (Phase 28 fix).
`generate_plan(...)`	Entry point. Selects pool, builds each workout day, labels rest days.
`_build_session(...)`	Picks exercises round-robin across the day's muscles, then adds or trims sets until the chosen length (minus 10 min) is filled within experience bounds.
`_prescribe(ex, goal)`	Applies `GOAL_PRESCRIPTION` to reps and rest; timed holds (reps <= 1) are untouched.
`select_days_to_regenerate`, `regenerate_days`	Injury-aware rebuild: days already logged this week are locked; injured muscle groups are removed from targets; an emptied day falls back to the goal's top groups ("(Adjusted)") or becomes a rest day.
`get_logged_day_numbers_this_week`, `_resolve_local_today`	Uses the user's local date (`client_date`, default UTC+8). The older UTC version could lock the wrong weekday near midnight (Phase 28 fix).

Why rule-based? (explain this carefully)

The proposal calls the planner a **hybrid** of a rule-based engine and a scikit-learn model. In the final system the plan itself is rule-based and the learned model is the **fatigue predictor** that adjusts it. A rule-based generator is deterministic, explainable ("why this exercise?" has an answer), unit-testable, and needs no labelled training data, which you did not have. A learned recommender would need user history you could not collect in one semester. [C]

Bugs found by code review and fixed (Phase 28): your best "maturity" story

Bug	Measured effect	Fix
"Arms" treated as one group	92% of Pull days contained tricep dips or presses; Push days often had no shoulder work	`_target_group` splits biceps/triceps; round-robin across the day's muscles; focus areas cannot add off-split work.
Session length ignored	A 60-minute choice produced about 29-minute sessions	`_build_session` fills the requested time less 10 minutes, within experience bounds.
Goal had no effect on the prescription	Four goals gave equivalent plans	`GOAL_PRESCRIPTION` shifts reps and rest per goal.
Plan reset could leave no active plan	Old plan deactivated before the new one existed	Retire old plans in the same batch that publishes the new one.

Worked example: how long is this session?

Intermediate, 45-minute session, push day. Working time = sum over exercises of `sets x (45 s + rest)`. Five exercises x 3 sets x (45 + 75 s) = 1,800 s = 30 minutes. Add 10 minutes for warm-up and cool-down = 40 minutes. The generator is 5 minutes short, so it adds a set or an exercise (up to the intermediate maximum of 7 exercises and 5 sets) until the total reaches about 45. Try the same arithmetic for a 30-minute beginner.

Tests and what to say about them

- `backend/tests/test_plan_generator.py` (144 lines) and `test_regenerate_plan.py` (305 lines) test split choice, session length, goal prescription, locked days and injury fallbacks. Open them and run `pytest -v` to see which behaviour each name describes.
- Be precise: before Phase 28 there were no tests for the plan generator; these were added with the fixes (doc).
- Library facts: backend pool 326 exercises (126 hand-written plus 200 generated from the dataset); Flutter library 626. The two lists are separate (a known drift risk) and joined by exercise name.

Viva questions for Module 5

- Is this AI? (Rule-based planner plus an ML fatigue model; say so plainly. The AI claim is the adaptation loop and the pose pipeline.)
- How does the plan respond to the goal? (Only reps and rest, not exercise selection. Say it before they find it.)
- What if the user has no equipment? What if every exercise for a day is injured?
- How do you know the generator is correct? (Unit tests; Phase 28 review found 4 bugs; on-device REC tests found two more, below.)

Known weaknesses found on device (Chapter 5 log)

• REC-03: after an injury the rebuild refilled **all** unlogged days to 60 minutes; weekly sets rose from 69 to 105, chest 15 to 50 sets (vs the app's own MRV of 20), and manual edits were lost.

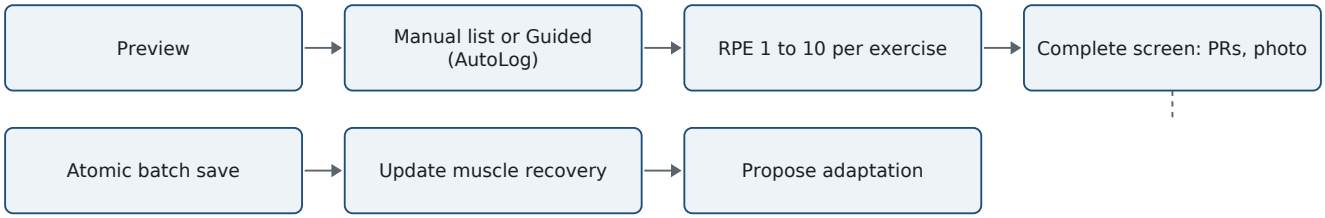
• REC-04: after recovery, the Friday legs day stayed glute-only because restore happens only via a full rebuild. Check whether either is fixed in later commits; if not, list as a limitation.

Teach-back task

• Generate a plan on paper: beginner, 3 days, full gym, 45 minutes, goal muscle gain. Which split, how many exercises per day, which tiers pick them? Then check your answer against the real output in the app.

MODULE 6 The workout session, logging and the recovery model

Where a plan becomes data. Everything the adaptive engine knows comes from what is saved here, so the save path and the recovery calculation must be understood exactly.



Read in this order

File (lines)	What to look for
`workout_preview_screen.dart` (304)	Chooses Manual or Guided mode.
`workout_active_screen.dart` (2226)	Manual logging: sets, reps, weight, per-exercise RPE slider, rest timer, PopScope guarding accidental back, finish early with confirmation.
`auto_log_screen.dart` (2092)	Guided mode as a phase machine: ready -> active -> resting -> rpe; optional camera rep counting.
`active_session_state.dart` (57)	`ExerciseSessionState` including `rpeRated` (only rated exercises feed average RPE).
`workout_log_service.dart` (549)	`saveWorkoutLog` (atomic batch, 15 s timeout, retry reuses the same `logId`), `updateMuscleRecovery`, `getWeightHistory`, `getLastTrainedDate`.
`workout_complete_screen.dart` (885)	Summary, PR detection, `_runAdaptation` which builds the six ML features.
`schedule_matcher.dart` (97)	`resolvedDayForDate`: the single place that answers "what is scheduled on date D" (override beats weekday template).
`muscle_recovery_constants.dart` (71)	`kSecondaryMuscleWeight`, `kGranularToBroadMuscleGroup`, MRV table.

The save path (why it was rewritten in Phase 28)

- Before: log, exercise logs, feed copy, counter and recovery were separate writes, so a failure could leave half a session.
- Now: **one atomic batch**. If the server has not acknowledged within 15 s the save is treated as queued: Firestore holds it locally and syncs later, and the user is told. A retry reuses the same `logId`, so duplicates are impossible. Test WRK-14 proved it: airplane mode on, finish a set, summary shown offline, airplane mode off, exactly one log on the server.
- `completion_rate` used to be always 1.0 because the user had to finish every set. Now the user can finish early; `totalSetsPlanned` and `completionRate` are stored. This repaired a degenerate ML feature.

The muscle-recovery model (explain the formula)

Piece	Rule (doc)
Window	Recomputed from scratch over the last 7 days of logs after every save (not an incremental counter).
Credit per set	Primary muscle 1.0; each distinct secondary group 0.5 (`kSecondaryMuscleWeight`).
Score	`fatigueScore = clamp(weightedSets / MRV, 0, 1)`
MRV (weekly sets)	Back, Chest, Legs, Glutes 20; Shoulders, Arms 16; Core 12; fallback 16 (Israetel et al. volume landmarks, per your citations).
Rest advice	`recommendedRestDays` = 2 if score < 0.5, else 3.
Volume	Weighted sets x reps x weight; bodyweight sets add reps only.

Worked example. In the last 7 days you did 10 back sets (primary) and, in other sessions, 8 sets that list back as a secondary muscle. Weighted sets = 10 + 8 x 0.5 = 14. Back MRV = 20. Score = 14 / 20 =

0.70. Rest days = 3. In the adaptive engine, a score above 0.8 triggers a forced deload and below 0.5 lets the weekly trend drive progress (Module 9).

Why recompute instead of counters? [C]

An incremental counter drifts when a log is edited, deleted or synced late. Recomputing from a 7-day query is always correct and cheap at this scale. This pattern repeats in weekly summaries and log queries (engineering principles).

Viva questions for Module 6

- What is RPE and why does the app collect it per exercise instead of per session?
- What happens if the phone loses signal at the end of a workout?
- Why is the recovery score load-based and not time-based? (By design, REC-01: chest trained today can still be "ready" if its weighted load is low versus MRV. Be ready to defend or admit this.)
- How do you detect a personal record?
- What does the user lose if they never rate RPE? (Only rated exercises feed the average; if none are rated it falls back to 5.)

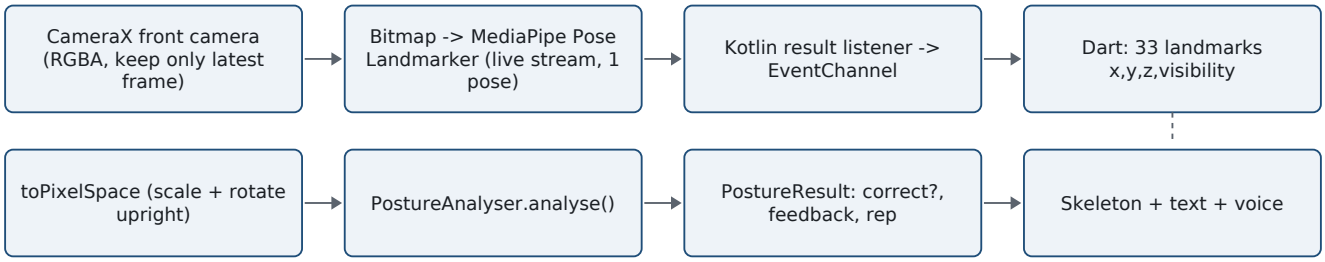
Teach-back task

• Trace one 4-set bench-press session from first tap to Firestore: list each document written, the fields, and the one function that writes them atomically.

MODULE 7 Posture detection: the headline feature

Spend about a third of your study time here. It is your most original work and the part with the most numbers an examiner can attack. Four sub-parts: pipeline, maths, analysers and thresholds, evaluation.

7a. The pipeline from camera to feedback



File (lines)	What to look for
`android/.../PoseDetectorHandler.kt` (219)	`setupMediaPipe`, `startCamera`, `processFrame(imageProxy)`, `handlePoseResult`, `sendEvent/sendError`, `cleanup`. Confidence thresholds 0.5; one pose; live-stream mode. Latest commits fixed MediaPipe timestamps and surfaced camera failures.
`MainActivity.kt` (80), `CameraPreviewFactory.kt` (32)	Registers channels `com.example.rakan/pose_landmarks` (events), `/permissions` (camera permission) and `/camera_preview` (platform view).
`pose_service.dart` (64)	`getLandmarkStream(exerciseType)` returns a Dart `Stream` and wraps native errors in `PoseStreamException`.
`pose_detection_screen.dart` (847)	Countdown overlay first; analyser is only called after it ends so hysteresis state is not polluted. Skeleton painter uses normalised coordinates; the analyser uses pixel coordinates.
`form_check_hud.dart`, `pose_countdown_overlay.dart`, `voice_coach.dart` (198)	How a verdict becomes red/green skeleton, text and spoken cues.

Why on-device, and why Android only?

• On-device: no video leaves the phone (privacy), no server cost, no network latency (doc).

• Android only: the MediaPipe Tasks Vision SDK is called natively from Kotlin through channels. The x86 emulator cannot load `libmediapipe_tasks_vision_jni.so` (test log: UnsatisfiedLinkError), so every camera test was done on the physical S20 FE.

7b. The maths you can be asked to do on paper

Joint angle. For three landmarks A, B, C (B is the joint), the angle is the angle between vectors BA and BC: `angle = arccos( (BA . BC) / (|BA| |BC|) )`, clamped and zero-safe, in degrees (`AngleCalculator.calculateAngle`).

Worked example (knee). Hip (300, 400), knee (300, 600), ankle (400, 800) in pixels. BA = hip - knee = (0, -200). BC = ankle - knee = (100, 200). Dot product = 0 x 100 + (-200)(200) = -40,000. |BA| = 200. |BC| = sqrt(100^2 + 200^2) = 223.6. cos = -40,000 / (200 x 223.6) = -0.894, so the angle is **153.4 degrees**: nearly standing, just under the 160 degree "standing" rule.

Worked example (elbow). Shoulder (0,0), elbow (0,100), wrist (100,100). BA = (0,-100), BC = (100,0). Dot = 0, so the angle is exactly **90 degrees**, the push-up depth cue.

Why pixel space matters (Phase 28 bug). MediaPipe returns x and y as fractions of frame width and height. On a 640 x 480 frame one unit of x is longer than one unit of y, so angles computed on raw fractions are skewed, up to about 16 degrees near 90 degrees (doc). `toPixelSpace()` multiplies by width and height first.

Why rotate (Phase 29). Joint angles do not change under rotation, but **trunk lean and shin angle are measured against gravity**, so the frame must be rotated upright first (270 degrees for the front camera in portrait). `angleFromVertical(upper, lower)` is the gravity-referenced measure. Example:

shoulder (330, 200), hip (300, 400): offset (30, -200), lean = atan(30/200) = **8.5 degrees**.

Visibility. Landmarks below visibility 0.5 are ignored. In a side-on view the far limb is occluded and MediaPipe guesses it, so each analyser picks the visible (near) side instead of requiring both sides.

7c. Rep counting and judgement: `_RepTracker`

A rep is **rest zone -> work zone -> back to rest zone** (hysteresis). While the rep is in progress the tracker records the primary angle's extreme and the worst value of each fault measure. When the rep ends it is judged **once**, at its extreme, and a `VALIDATION|...` line is logged for the study. Live faults (e.g. leaning too far right now) turn the skeleton red immediately; depth faults are shown after the rep as "Rep N: too shallow".

Worked trace (squat). Knee angle per frame: 172, 165, 140, 115, 95, 78, 92, 120, 150, 168. Counting threshold is 110 degrees, so the work zone starts at 95. The extreme is 78. The standing threshold is 160, so the rep closes at 168. Verdict: 78 <= 80 means depth is correct (assuming trunk lean <= 57). If the extreme had been 95, the rep is **counted but flagged "too shallow"**.

> **Why two thresholds (Decision #62)**
>
> • Before Phase 29, a rep only counted if it was correct, so a bad rep vanished and could never be a false negative. The confusion matrix was structurally one-sided. A looser **counting** threshold plus a stricter **correctness** threshold makes bad reps countable and scoreable.

7d. Thresholds and the evidence behind them

Design rule (Decision #61): threshold = published reference value + 10 degrees measurement tolerance; for joints judged near full extension (about 180) the tolerance is 20 degrees. The tolerance is one named constant (`FormThresholds.tolerance`), so you state it once and the study tests it. Values with no published source are labelled **engineering estimates** in the code.

Exercise family	Rule (doc: POSTURE_THRESHOLDS)	Source
Squat (bodyweight, barbell, goblet, front, band)	Counted at knee <= 110. Correct depth <= **80**. Standing >= 160. Trunk lean (from vertical) <= **57**. No "too deep" fault.	Parallel squat about 60 to 70 degrees inner knee (Rojas-Jaramillo et al., 2024) + 10. Lean 37.8 +/- 9.7 at parallel (Graber et al., 2023): mean + 2 SD = 57.
Lunge / split squat	Counted <= 120. Correct front knee <= **100**. Trunk lean <= **30**. Front leg = leg whose shin is closer to vertical.	Farrokhi et al. (2008): lunge about 70 inner angle; trunk-forward adds about 20.
Deadlift	Hip <= 100 to count the hinge, >= 165 lockout. **No back-rounding check.**	No spine landmarks in the 33-point model.
Romanian deadlift	Counted hip <= 125; lockout >= 165; knee stays >= **155**; depth hip <= **110** (estimate).	Physiopedia (n.d.); depth is an engineering estimate.
Push-up family	Counted elbow <= 120; correct <= **100**; body line (shoulder-hip-ankle) >= **160**. Pike push-up flags hips > **120** (estimate). Bench press judges elbow depth only.	FitnessGram 90 degree standard + 10 (Topend Sports, n.d.).
Shoulder press	Bottom <= 100; top counted >= 145; lockout correct >= **160**; left/right asymmetry <= **20**.	180 - 20 near-extension tolerance; asymmetry = 2 x 10 tolerance.
Curl family (side-on)	Counted <= 70; correct <= **50**; extended >= 140; upper-arm swing <= **30**.	40 to 160 ROM used by MediaPipe curl counters; Chua et al. (2024).

Self-corrections from the literature (tell this story, it shows rigour)

- Original squat rule called 80 to 100 degrees "good" and below 80 "too deep". That window is a **half squat**; a proper parallel squat was marked wrong. Corrected using Rojas-Jaramillo et al. (2024), and the "too deep" fault was removed because 13 of 15 studies found deep squats safe for healthy knees.
- Push-up depth was never judged (any counted rep passed). Now <= 100.
- Pike push-ups and bench press used the straight-plank test they always fail. Fixed.
- Bulgarian split squats could never count a rep (rear knee chosen as the front knee). Fixed with the shin-vertical rule.

- Curls were analysed front-on where the forearm is foreshortened. Now side-on.
- Left out deliberately (Decision #64): Arnold press, walking lunge, jump squat, archer push-up, incline/concentration curl, because the movement cannot be seen reliably in 2D.

7e. The accuracy study (Objective 2): design, results, honest reading

Design (doc: tools/posture_study). Controlled-condition study. The app logs what it **predicted**; accuracy needs what the tester **actually did**, which the app cannot know. So before each set you tell the tester to do deliberately correct or deliberately faulty reps and type the label into `capture.py`, which stamps every incoming `VALIDATION|...` line with that ground truth (`x` flips a label if the tester did not do what was intended). Positive class = **incorrect form** (the system's job is to catch bad form). Anonymised tester IDs T01 to T05. Metrics: accuracy with 95% Wilson interval, precision, recall, specificity, F1, rep detection rate (missed reps are reported separately).

Exercise	Reps	TP	TN	FP	FN	Accuracy (95% CI)	Precision	Recall	Specificity
Push-up	150	80	70	0	0	100% (97.5 to 100)	1.00	1.00	1.00
Squat	150	69	68	9	4	91.3% (85.7 to 94.9)	0.885	0.945	0.883
Overall	300	149	138	9	4	95.7% (92.7 to 97.5)	0.943	0.974	0.939

Reading the squat matrix. FP = a good rep the app flagged as bad (9 of 77 good reps). FN = a bad rep the app passed (4 of 73 bad reps). Recall 69/73 = 0.945; specificity 68/77 = 0.883; accuracy (69+68)/150 = 0.913. Wilson interval is used instead of the simple normal approximation because n is small and the push-up accuracy is exactly 1.0, where the simple formula gives a zero-width interval.

Why push-up reached 100%, and why you must say it before they do

• The faults were **exaggerated** (the log: push-up body line 55 to 145 degrees, squats stopping at 95 to 110) so classes are well separated. Borderline faults were not tested. Real-world accuracy may be lower (workbook, 'Exaggerated faults').

• Tester balance: T01 push-up all 30 reps wrong, T03 push-up all 30 correct, T04 squat all 30 wrong. Classes are balanced overall (push-up 70/80, squat 77/73) but per-tester accuracy is not comparable.

• Fault types were not recorded per set, so accuracy cannot be reported per fault.

• Possible mislabels: a few 'shallow' squat reps went below parallel (e.g. 64, 64, 41 degrees); the app said correct, so they count as misses.

• Two MediaPipe tracking glitches produced impossible readings (e.g. knee 15.8 degrees). Report as a real-world failure mode.

Data-handling issues the workbook itself flags (resolve these first)

• **What happened (your explanation):** in T02 squat set 2, reps 1, 2, 4 and 7 (knee 83.6, 81.2, 81.1, 81.4 degrees) were changed after the session because they could not be corrected live. **Be careful:** the changed column is `predicted_correct`, the app's own verdict, not `ground_truth_correct`. A late correction to ground truth is acceptable. A change to the app's output is not, because it turns 4 false positives into matches.

• **How to say it:** "Four predicted-verdict cells in one set were edited by hand after the session. I disclose this, and every figure I report recomputes the verdict from the logged angle using the code rule, so the edits do not affect the result. Recomputed, those four reps are false positives, and they are included in the 91.3%." Never describe the edit as correcting a label.

• **For the future:** label fixes go in the ground-truth column (the x command in `capture.py`). The app's verdict is never edited.

• **5 net observer flips, all moved toward the app's verdict.** Fine if each was based on what you saw; not fine if based on the app. The sensitivity table gives squat 88% (ignoring all flips), 91.3% (main) and 94% (verdicts as logged).

• Say: "Under all three treatments squat accuracy stays above the 75% target; the push-up result is 100% in every treatment." Then explain each flip from your memory of the session. If you cannot, say so.

• Your UAT form recorded consent for all 20 participants. Check whether the 5 posture-study testers also gave recorded consent. Ethics is a likely question for any study with human participants. **CHECK.**

Rule-based vs trained classifier: your argument [C]

- A rule gives **specific, actionable feedback** ("knees past 80 degrees, go lower"). A classifier gives a label and a probability.
- No dataset had labelled faults for your 31 exercise variants and your camera setup. Kaggle landmark datasets in the proposal label the *exercise*, not whether the rep was correct.
- Rules are traceable to papers, which is what the supervisor asked for (no arbitrary thresholds).
- Your evaluation applies the standard classification metrics (confusion matrix, precision, recall, F1) to the rule system, which is a legitimate ML evaluation methodology.
- Trade-off: thresholds are population averages, not personalised to body proportions; MediaPipe is trained on diverse data (the proposal's question) but you did not test across body types or lighting.

Tests for this module and what each proves

Test file	Proves
`angle_calculator_test.dart` (4 tests)	Known triangles give 90, 180 etc.; zero-length safe.
`posture_variants_test.dart` (23 tests)	Every `hasPoseDetection` exercise maps to an analyser (factory test); variant thresholds; Bulgarian split squat counts reps.
`voice_coach_test.dart` (15), `pose_service_test.dart`	Cue selection and stream error handling.
Accuracy study (physical device)	Real MediaPipe landmarks on a real human, not synthetic input.

Viva questions for Module 7

- Why 2D? What do you lose? (Depth ambiguity; side-on view chosen where movement is sagittal so projection error is smallest.)
- Where does 80 degrees come from? Where does 57 come from? Where does 20 come from?
- Why the 10 degree tolerance? What happens if you set it to 0 or 20? (Run the study with a different constant: it is one named value.)
- What is a false positive in your system, and which is worse for a user: a false positive or a false negative? (Argue it. Injury risk suggests missed bad reps matter; trust suggests false alarms hurt too.)
- How do you know your ground truth is right? (Observer label before each set; video optional; flips logged.)
- What about different body types, clothing, lighting, camera height? (Not tested; limitation; MediaPipe diversity is a claim from the proposal that you did not verify.)
- Why is the deadlift not evaluated? (No spine landmarks; analyser has no fault check; correct-only rep counting could be shown.)

Teach-back tasks

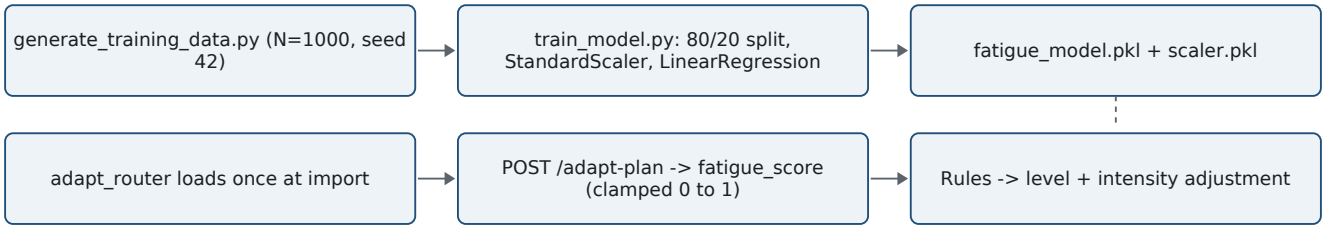
• On paper, compute the elbow angle for shoulder (0,0), elbow (40,90), wrist (130,60). Say whether it passes the push-up depth rule and why.

• Rebuild the squat confusion matrix from the four numbers 69, 68, 9, 4 and derive accuracy, precision, recall, specificity without a calculator.

• Explain in 90 seconds why a counted-only-if-correct rep design cannot produce false negatives.

MODULE 8 The fatigue model (machine learning)

The only trained model in Rakan. Examiners will test whether you understand what it learned, what it did not, and why it is still useful.



Read in this order

File (lines)	What to look for
`backend/ml/generate_training_data.py` (56)	Experience mix 40% beginner, 40% intermediate, 20% advanced; base RPE by experience (5.0, 6.5, 7.5) + N(0, 1.5); duration N(50, 15) clipped 20 to 120; 3 to 7 exercises; completion falls as RPE rises; then the label formula.
`backend/ml/train_model.py` (69)	FEATURES list, LABEL, train/test split, scaler, fit, MAE and R-squared, `joblib.dump`.
`backend/routers/adapt_router.py` (132)	`AdaptRequest` with range validation (RPE 1 to 10, completion 0 to 1), `_apply_adaptation_rules`, fallback to `avg_rpe / 10` if model files are missing.
`workout_complete_screen.dart` `_runAdaptation`, `adapt_service.dart` `predictAndAdapt` (line 616)	How the six features are built on the client and sent.
`backend/tests/test_adaptation_engine.py`	Rules and boundaries.

The model in one table

Item	Value (doc)
Features (6)	avg_rpe, max_rpe, session_duration (min), exercises_count, completion_rate (0 to 1), experience_level (0, 1, 2)
Label formula	`fatigue = 0.35 (avg_rpe/10) + 0.25 (max_rpe/10) + 0.15 (duration/120) + 0.15 (1 - completion) + 0.10 exp_penalty + N(0, 0.05)`, clipped to [0, 1]. exp_penalty: beginner 0.30, intermediate 0.15, advanced 0.
Training set	1,000 synthetic rows; label counts 253 low (< 0.4), 713 medium, 34 high (>= 0.7)
Performance	MAE 0.0404, R-squared 0.829 (recomputed by the overview's author from the committed CSV, same seed). **Re-run** `train_model.py` **and note your own numbers.**
Coefficients (standardised)	avg_rpe +0.055, max_rpe +0.052, duration +0.018, exercises -0.0003, completion -0.014, experience -0.011
Fallback	If the model files are missing: `avg_rpe / 10` (RPE-based regulation, Zourdos et al., 2016)

Rules that turn the score into action (single source of truth)

fatigue_score	Level	Intensity change	Cited basis (doc)
> 0.7	high	**-17.5%** sets and reps	Bell et al. (2025), reactive deload
0.4 to 0.7	medium	**0%** (maintain)	ACSM (2026): RPE 6 to 7 and completion 70 to 90%
< 0.4	low	**+10%**	ACSM (2026), NASM: 5 to 10% progressive overload

Worked examples (compute these by hand)

Session	Calculation	Score	Result
Intermediate, avg RPE 8, max 9, 60 min, completion 100%	0.35 x 0.8 + 0.25 x 0.9 + 0.15 x 0.5 + 0 + 0.10 x 0.15 = 0.28 + 0.225 + 0.075 + 0 + 0.015	0.595	medium, 0%
Beginner, avg RPE 9, max 10, 90 min, completion 60%	0.315 + 0.25 + 0.1125 + 0.06 + 0.03	0.7675	high, -17.5%
Advanced, avg RPE 5, max 6, 40 min, completion 100%	0.175 + 0.15 + 0.05 + 0 + 0	0.375	low, +10%

The model approximates these numbers; it will not equal them exactly because of the noise term and the learned weights.

Applying the adjustment to the plan

For each future exercise of the affected muscle group: `sets = clamp(round(sets x (1 + adj)), 1, 6)` and `reps = clamp(round(reps x (1 + adj)), 3, 20)`. Example for a 4 x 10 exercise: at -17.5% sets = round(3.3) = 3, reps = round(8.25) = 8; at -42.5% (deload) sets = 2, reps = 6; at +10% sets = round(4.4) = 4, reps = 11. **Weight is never changed**, which is why plateau detection needs no filter (Decision #57).

The honest reading of R-squared = 0.83 (rehearse this)

- The labels come from a hand-written linear formula, and a linear regression recovers a linear formula. So 0.83 shows the model **reproduces the fatigue heuristic**, not that it predicts real-world fatigue. Present it exactly like that.
- Why synthetic data? There was no labelled dataset of app-measured RPE and fatigue for your user group, and collecting one safely needs weeks of ethical data collection. The proposal listed the Kaggle AFR-1000 athlete fatigue dataset; **CHECK and prepare your own reason for not training on it** (do the dataset's columns match what the app can measure?).
- Why linear regression? Interpretable coefficients, only six inputs, and your project rule is fundamentals before advanced AI. A tree-based model would fit the same formula no better and add opacity.
- The **negative experience coefficient** (-0.011) looks wrong. It happens because more experienced users have higher base RPE in the generator, which is collinear with the penalty term that favours beginners. Be ready to say this.
- Real value of the ML step in the system: it lets one number blend several signals (effort, peak effort, duration, completion, experience) into a smooth score, and the thresholds are cited and tested separately.

Two bugs that made the model's inputs wrong (found in Phase 28)

- `experience_level` was always 0: the app read `profile['experience']`, but the stored key is `experienceLevel`. Every user was sent as a beginner. Fixed.
- `completion_rate` was always 1.0 because finishing required every set. Fixed by allowing early finish. Unrated RPE had also counted as 5; now only rated exercises feed the average.
- Lesson for the viva: unit tests on the backend could not catch these because the backend was correct; the fault was in the client's feature construction (an integration bug). That is why the on-device tests matter.

Viva questions for Module 8

- Is this real AI? Why linear regression? What would you change with real data?
- What does R-squared 0.83 tell me? What does it not tell me?
- Why clamp the prediction? Why thresholds 0.4 and 0.7?
- What if a user always enters RPE 10 (or always 1)? (Garbage in, garbage out; completion rate and duration partly offset; future: detect inconsistent rating.)
- How would you evaluate whether adaptation reduces overtraining? (Design: 4-week pilot, compare RPE trend, completion, drop-out and self-reported soreness with and without adaptation. Not yet

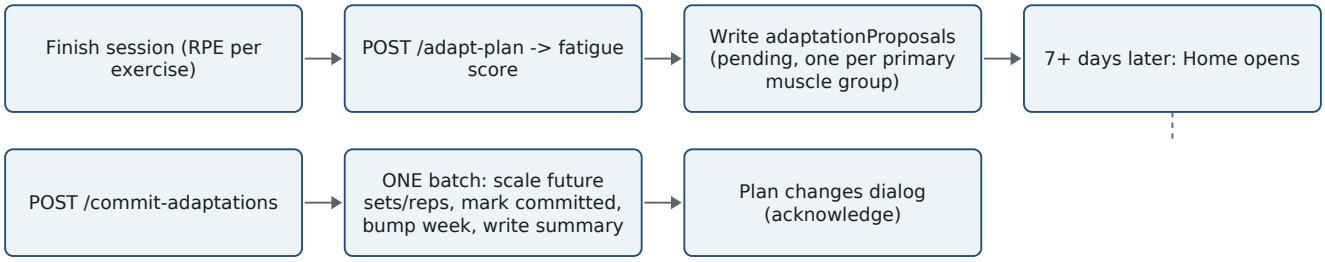
done.)

Teach-back task

• Open `train_model.py`, run it, and explain every line of its output to an imaginary friend who knows no ML. Then change one coefficient in `generate_training_data.py` and predict, before running, how R-squared and the coefficients change.

MODULE 9 The adaptive engine: propose, commit, tiers, missed days, plateau

The core contribution. It is also the largest body of logic with unit tests, so you can point to evidence for most claims. Learn it as one story: a session proposes, a week commits, one priority rule resolves conflicts.



Why propose-then-commit? (Decision, doc) Adjusting after every workout makes the plan lurch on a single bad day and makes bugs harder to trace. Proposals are cheap and reversible; one weekly commit applies the adjustment to future days atomically and creates a human-readable summary. The user always sees **what changed and why** (`reason` strings carry citations).

Read in this order

File (lines)	Symbols to find
`backend/services/adaptation_engine.py` (203)	Constants (`RECOVERY_FORCED_DELOAD_THRESHOLD = 0.8`, `RECOVERY_WELL_RECOVERED_THRESHOLD = 0.5`, `FORCED_DELOAD_FLOOR = -0.175`, `SCHEDULED_DELOAD_ADJUSTMENT = -0.425`, `RETURN_FROM_BREAK_ADJUSTMENT = -0.175`, `DETRAINING_THRESHOLD_DAYS = 28`, `TREND_INCREASING_THRESHOLD = 0.05`, `TREND_DECREASING_THRESHOLD = -0.10`), `compute_trend`, `resolve_adjustment`.
`adaptation_engine_router.py` (73)	`/commit-adaptations`: trend plus per-proposal decisions.
`adapt_service.dart` (693)	`buildSessionProposals`, `buildReturnFromBreakProposal`, `computeMissedDays`, `computeValidRescheduleDays`, `resolveMissedDayAsRescheduled/Skipped`, `detectPlateau`, `predictAndAdapt`.
`weekly_summary_service.dart` (419)	`checkAndGenerateWeeklySummary`, `_generateSummary`, `_applyAdjustmentToMuscleGroup`, `acknowledgeChanges`.
`missed_day_dialog.dart` (549), `plan_changes_dialog.dart` (290)	UI for choices and for the weekly changes; Decision #51 (state keyed by item identity, not list index).
`test/adapt_service_test.dart` (737 lines, 38 tests), `backend/tests/test_adaptation_engine.py`	Boundary tests; read the test names as a specification.

Priority resolution: `resolve_adjustment` (learn this table cold)

Tier	Condition	Result	Why this order
0 scheduled_deload	Week number % 4 == 0	**-42.5%**, overrides everything	Planned recovery block every 4th week (mesocycle). **CHECK:** confirm the citation for -42.5% in your handover.
0.5 return_from_break	Trigger is a skipped session	Gap <= 28 days: **0%**. Gap > 28 days or unknown: **-17.5%**	Short-term detraining retains adaptations; beyond 4 weeks it does not (Mujika and Padilla, 2000; Encarnacao et al., 2022). Your note: the 2-skip trigger is justified on **behavioural/adherence** grounds (Gjestvang et al., 2023), not physiology.
1 forced_deload	Muscle recovery score > 0.8	`min(session_adj, -17.5%)`	Near or over MRV: safety first.
2 weekly_priority	Recovery < 0.5	`max(session_adj, trend_adj)`	Well recovered: the longer-term trend may drive progression.
3 session_priority	0.5 to 0.8	Session fatigue adjustment	Acute, safety-relevant signal wins in the middle band.

Trend. `pct = (current - mean(past)) / mean(past)`. Above +5% is increasing (+0.05); below -10% is decreasing (-0.10); otherwise stable (0). No history or a zero baseline gives `insufficient_data`. Example: this week's volume 12,000 kg, past weeks 10,000 and 11,000 (mean 10,500): pct = +14.3%, so increasing.

Worked examples for `resolve_adjustment`

Situation	Tier fired	Final
Week 4, any muscle	scheduled_deload	-42.5%
Skip proposal, 20 days since last trained	return_from_break (<= 28)	0%
Skip proposal, 40 days since last trained	return_from_break (> 28)	-17.5%
Recovery 0.85, session says +10%	forced_deload: min(+10%, -17.5%)	-17.5%
Recovery 0.30, session 0%, trend +5%	weekly_priority: max(0, +5%)	+5%
Recovery 0.65, session -17.5%, trend +5%	session_priority	-17.5%

Missed-day flow (Phases 24 to 26)

- **Detection** (`computeMissedDays`, pure): scan back at most 30 days from yesterday; a scheduled workout day with no log and no resolution document is missed, per **muscle group + date** (key `"{group}|YYYY-MM-DD"`). Bounded by the plan's `generatedAt` so a new plan does not "miss" days before it existed (Phase 26 bug fix).
- **User choice:** Reschedule or Skip entirely. UI groups by date but resolves per muscle group.
- **Reschedule rules** (`computeValidRescheduleDays`): candidate date must be (1) after the missed date and after today, (2) at least 48 hours after the group's actual last-trained date, (3) before the group's next scheduled occurrence, (4) an empty day (rest/unscheduled, no override). Writes a one-off `scheduleOverrides` doc and a resolution `rescheduled`; **no adaptation proposal**.
- **Skip:** writes resolution `skipped` and a `trigger: skip` proposal (idempotent: one pending skip proposal per group), capturing `priorLastTrainedDate` now; `daysSinceLastTrained` is filled when the group is next trained.
- **Collision rule (Decision #44):** a skip proposal suppresses the ordinary session proposal for the same muscle group, so one disruption is not counted twice. **#45:** skip proposals have a null fatigue score.
- **Expiry:** missed days older than 7 days auto-resolve as skipped on app open.

- **Worked example:** Push last trained Monday, missed Wednesday. Thursday is a rest day, more than 48 hours after Monday, and before next Monday's Push. All four rules pass, so Thursday is offered. Friday would also be valid if empty. Tuesday is invalid (before the missed date).

Plateau detection (Phase 27, Decisions #57 to #60)

`AdaptService.detectPlateau(sessionMaxWeights, n=4, thresholdPct=0.02)` is a pure function. Four session-to-session transitions need **five** values (N+1, Decision #59). A plateau means **no** transition improved max weight by at least 2% (inclusive). A previous weight of 0 or less is skipped to avoid divide-by-zero.

Five session max weights	Reasoning	Plateau?
40, 40, 40, 40, 40	Every transition 0%	**Yes**
40, 40, 40, 40, 41	Last transition +2.5% >= 2%	No
40, 40, 40, 40, 40.5	Last transition +1.25% < 2%	**Yes**
40, 40, 40, 40, 40.8	Exactly +2% (inclusive)	No
40, 40, 40, 40	Only 4 values	No (insufficient history, not plateau)

- **Why it is not a signal (#57):** signals prescribe (change sets/reps); plateau only observes and never writes. Keeping the line explicit prevents overstating what the system does.
- **Why on-demand (#58):** no persisted `plateauFlags` collection, so no resolution/idempotency state machine; the flag is reconstructable by replaying logs.
- **Known limit (#60):** a long calendar gap looks like stagnation. Accepted and documented, same discipline as #56 (client-clock dependency of the return-from-break signal).
- **Why 2%:** below the 2.5 to 5% progressive-overload range, so sub-threshold progress reads as no meaningful improvement. Why N = 4: matches the project's 4-week evaluation window.
- A bug found while fixing Phase 28: the history fetch defaulted to 4 sessions, so plateau could never fire (it needs 5). Now 5.

Why the pure-function split matters for testing

`computeMissedDays`, `computeValidRescheduleDays` and `detectPlateau` are **static pure functions**: data in, answer out, no Firestore. The Firestore reads live in separate methods (`findMissedDays`, `getRecentSessionMaxWeights`). That is why the logic has dozens of boundary tests that run in milliseconds without a database. Say this deliberately; it is a design-for-testability argument that examiners like.

Verification you can cite

- Unit tests: missed-day detection, reschedule window, expiry partition, skip resolution, 10 plateau tests; backend 28-day boundary (28 days gives 0%, 29 gives -17.5%).
- On device (Phase 26): Scenarios A to E for the missed-day flow; Signal #7 tested by moving the device clock, which is event-driven so a real workout must be completed after the jump. Side effect worth knowing: a manually set clock caused a Google Sign-In UNKNOWN error until automatic time was re-enabled.
- Seeded scenarios: `tools/demo_seed/seed_adaptive_demo.py` builds accounts with history to demonstrate deloads and plan changes (Chapter 5: demo.deload account shows 'DELOAD WEEK: Adjusted -43%' on the Records tab).

Viva questions for Module 9

- Why weekly and not after each session? Why one batch write?
- What if two signals disagree? (Tier order, with safety first.)
- Where does the -17.5% come from? the +10%? the 28 days? the 48 hours? the 2%? (Have a one-line citation or a stated engineering rule for each.)
- What stops the plan from shrinking forever? (Clamps: sets 1 to 6, reps 3 to 20; trend lets progress resume when well recovered; weight is never reduced by the engine.)

- How do you know it improves outcomes? (Not yet shown; unit and device tests verify the logic does what the design says. Effectiveness needs a user trial.)
- What does Signal #7's client-clock dependency mean for cheating? (A user can change the phone clock to alter the gap; server-side timing validation is future work, Decision #56.)

Teach-back task

• Without notes, draw the five-tier decision tree and run the six worked examples through it. Then explain the difference between a skipped muscle group and a rescheduled one in terms of the proposals written.

MODULE 10 Coach, social, notifications and settings (lighter)

Lower priority for marks, but a hole here is embarrassing. Know what each does and one design reason per feature.

Area	Files (lines)	What and why
Coach stats	`training_stats.dart` (405), `stats_report_tab.dart` (1327), `coach_screen.dart` (1803)	KPIs (workouts, volume, duration), weekly trend (`fl_chart`), consistency ring, muscle-focus distribution, per-exercise progression and plateau notice. Past bug: stats read a non-existent `isRestDay` field instead of dayType (same class of bug as an earlier Phase 14 fix). Fail-closed fix: `dayType == 'workout'`.
Recovery map and injuries	`log_injury_screen.dart` (913), `injury_service.dart` (202)	Tap a body region, choose expected recovery days; injury maps to muscle groups (head and neck deliberately excluded); calls `/regenerate-plan` best-effort (errors swallowed, 90 s timeout); recovered injuries are ignored. Recovery check-ins and plan restore added in a later commit.
Weight and records	`weight_record_service.dart` (95), `all_records_screen.dart` (91)	Body-weight log (add, edit, delete) and PRs (`WorkoutPrRecord`). The adaptive loop does not use weight or BMI yet (backlog).
Social	`follow_service` (161), `public_profile_service` (123), `activity_interaction_service` (106)	Follow or request, search by username prefix, likes and comments on feed items, share sheet. Purpose: adherence and motivation, which ties back to the dropout statistic. Privacy by structure (Module 3).
Notifications	`notification_service.dart` (475)	Local notifications only; fixed ids 101 to 107 for weekday reminders and a base of 200 for per-day start-time reminders so one can be replaced without touching others; `timezone` + `flutter_timezone`.
Settings	`settings_screen.dart` (1023), `edit_*` screens, `account_deletion_service.dart` (178)	Profile, stats, equipment, goals (plan reset), private account switch, password, in-app deletion (Google Play requirement).

Viva questions for Module 10

- Why does a posture app have a social layer? (Adherence; the problem statement is dropout.)
- What data can a stranger see on a public profile? (Name, username, photo, and feed summaries without weights.)
- How are injuries used by the planner? (Excluded muscle groups, locked logged days, recovery reintroduction.)
- What does the app tell users about medical responsibility? CHECK: is there a disclaimer in the app? A fitness app giving injury-related advice should say it is not medical advice.

Teach-back task

• Walk through what happens, step by step, from the moment a user logs a knee injury to the moment next week's leg day changes. Name the service, the endpoint and the document at each step.

MODULE 11 Testing and evaluation: why each test exists

The question behind every test is: what could be wrong that this test would catch, and what could still be wrong that it would not? Answer it for every row.

The testing strategy at a glance

Level	What	Why this level	Evidence
Unit: Dart	`flutter test`: adapt_service (38), posture_variants (23), voice_coach (15), injury_reminder (13), training_stats (10), schedule_matcher (5), angle_calculator (4), workout_plan_service (3), workout_screen (2), plus widget tests	Pure logic with boundaries: fast, deterministic, no device. Splits pure/IO on purpose.	105 passed in the 18 Oct run. **Re-run for a fresh count.**
Unit: Python	`pytest`: adaptation engine, plan generator, regenerate plan	Stateless rules API makes this cheap and exact.	Phase 28 reported 14 passing; later files add more. **Re-run.**
Security rules	25 scripted REST checks with two real accounts (A and B), `ch5_rules_test.py` (ST series)	Rules decide who reads what; only a real token proves it.	25/25 as expected.
System (emulator)	Chapter 5 cases by area: AUTH, ONB, HOM, WRK, SCH, REC, CCH, SOC, SET, LIB, IT	End-to-end user flows; finds integration and UI bugs unit tests cannot.	`tools/demo_seed/ch5/RESULTS_LOG.md`.
Device (physical)	S20 FE: posture, guided mode and other on-phone cases	Camera and MediaPipe do not run on the x86 emulator.	Your on-phone guide-sheet results (GDE and NP series in the log; check what each series covers).
Offline / integration	IT-10, WRK-14	Firestore cache and queued writes are claims that need proof.	No duplicate log after reconnect.
Accuracy study	5 testers, 300 reps, confusion matrices	Objective 2 needs measured accuracy, not unit tests.	Posture_Accuracy_Results.xlsx.
User acceptance test	20 participants, 22 to 23 Oct: SUS (10 items) + 8 feature items (F1 to F8) + 2 open questions, Google Form, consent recorded	Objective 1 is about satisfaction, and only users can measure it. SUS is a standard, comparable usability score.	SUS mean 85.1; F8 overall satisfaction 95% agree; all F items 94.4% agree.
Measurement	Cold start 52.37 s (warm 0.07 to 0.1 s)	Quantifies the cost of the free tier.	COLDSTART_output.txt.

Named techniques you are already using (say the names)

- **Boundary-value analysis:** 28 vs 29 days (return from break), exactly 2% (plateau), 4 vs 5 sessions (N+1), 48 hours (reschedule), 0.8 and 0.5 recovery bands.
- **Equivalence partitioning:** low, medium and high fatigue; tiers 0 to 3; push-up correct/incorrect classes.
- **Black-box vs white-box:** the Chapter 5 flows are black-box; the pure-function tests are white-box.
- **Regression testing:** after each phase the whole suite was re-run (Phase 27 handover records this explicitly).
- **Test-data seeding:** `seed_adaptive_demo.py` and the `ch5_*` scripts create repeatable histories, flagged `seededForCh5` so they are never mistaken for real data.

Defects the system tests found (Chapter 5 log): know the story and the status

Defect	Severity	Cause (from the log)	Status
Follow fails for accounts with no `isPrivate` field (SOC-02, D-24)	High	Rules error on missing field; client treats missing as public	CHECK `git log`
Logged day not protected from Move/Make rest (SCH-08)	Medium	Completion matched by date, not by workout	CHECK
Offline relaunch routes to 'Verify your email' (AUTH-02, D-28)	Medium	Network error treated as not verified	CHECK
Invisible success snackbars (AUTH-07, AUTH-14, D-23)	Low	Text colour equals background	CHECK
Template day adds 2 exercises, dialog promises 3 to 4 (SCH-06)	Low	Pool filter by equipment/focus	CHECK
Photo crop confirm unreachable on API 36 emulator (SET-03, D-26)	Low	Edge-to-edge toolbar under status bar	Recheck on physical phone
Injury rebuild overshoots volume and overwrites manual edits (REC-03, L-01)	Medium	Rebuild refills every unlogged day to 60 min	CHECK
Onboarding name not shown in Settings for email accounts	Low	Reads Auth `displayName`, never set by onboarding	CHECK

What the testing story proves

• You did not only test the happy path: a systematic emulator pass with named test IDs found real defects, you logged cause and evidence for each, and fixed or documented them. That is the strongest sentence you have in the evaluation discussion, **if** you can say which are fixed. Spend an hour with `git log` and the log to build that status column.

What your testing does not cover (say it first)

- No automated tests for Firestore rules (the ST checks are scripts), social features or most screens.
- UAT was a single session with 20 participants, probably a convenience sample (say who they were). It shows acceptance, not long-term adherence or outcomes. There was no long-term trial.
- Posture: controlled, exaggerated faults; 5 testers; two exercises; one phone.
- iOS is untested for pose; iOS and desktop scaffolds are default.

Viva questions for Module 11

- How do you know the app works? Which test would you trust most and which least?
- Which bugs did testing find? Give one that unit tests could never have caught.
- Why did you split pure logic from Firestore code?
- Why can the emulator not test posture detection?
- Explain your UAT. Who were the participants, how were they recruited, how long did they use the app, and why SUS? (Answer: 20 participants, SUS 85.1, above the commonly cited average of 68, plus 95% overall satisfaction.)

Teach-back task

• Pick any five test names from `adapt_service_test.dart`. For each, state the sentence of the design it protects and the bug it would catch.

MODULE 12 Limitations, future work and the viva game plan

A limitation you state calmly is a strength. A limitation the examiner discovers is a weakness. Rehearse these answers until they are boring.

Limitation table: honest answer and mitigation

Limitation	Honest answer	Mitigation / future work
Fatigue model trained on synthetic labels	R-squared 0.83 only shows it reproduces the heuristic.	Collect real RPE and outcome logs from a pilot; retrain; compare against the rules baseline.
Posture is 2D, single camera, no spine	Angle error and occlusion; back rounding not claimed.	Side-on set-up instructions per exercise; 3D landmarks or a second view; add spine proxies.
Small controlled accuracy study	300 reps, 5 testers, exaggerated faults, 2 exercises; some data-handling issues (flagged in the workbook).	Larger and blinded study, subtle faults, video-labelled ground truth, per-fault metrics, deadlift.
Backend endpoints unauthenticated	ST-14 demonstrated it; `uid` is trusted from the body.	Verify Firebase ID token server-side; rate limiting.
Client clock for return-from-break (#56)	A user can change the device date.	Server timestamps; Cloud-based validation.
Plateau false positive after long gaps (#60)	Detraining looks like stagnation.	Cross-reference break history.
Render cold start about 52 s	First request each session may be slow.	Pre-warm; move logic on device; paid hosting.
Two exercise libraries	Backend 326 vs client 626, joined by name.	Single source of truth (shared JSON or API).
Goal affects only reps and rest	Exercise selection ignores goal.	Goal-specific exercise priority; goal-change propagation (backlog).
Adaptation effectiveness not measured	Logic is verified; outcomes are not.	4-week controlled pilot (adherence, RPE trend, drop-out).
UAT is short-term and small	20 participants in one session; likely a convenience sample; self-reported.	Longer field trial with a broader sample; log actual usage.
Android-only pose; package id `com.example.rakan`	Prototype scope.	iOS native bridge; proper application id for release.
Base64 photos in Firestore	Chosen because Storage needs a paid plan.	Move to Storage if budget allows.

Future work (aligned with your backlog)

- Combine-with-existing-day reschedule option (needs a volume-stacking design first).
- Repeated-return escalation; weight and BMI trend feeding the adaptive loop; goal-change propagation.
- Server-side authentication and timing validation; range queries replacing the 30-log scan.
- Per-fault posture analytics; personalised thresholds by body proportion; more exercises.
- A real user study (UAT plus a 4-week adherence trial).

Ethics and responsibility (prepare a clear paragraph)

- Rakan is a fitness tool, **not medical advice**. It should tell users to stop if they feel pain and to see a professional for injuries. CHECK that the app says this.
- Data minimisation: video never leaves the phone; followers see summaries without weights; account deletion removes data.
- Participants: anonymised codes (P01 to P20, T01 to T05); consent recorded in the UAT form (check the posture testers too); no health data beyond exercise performance.

A 3-minute opening pitch (adapt to your own words)

1) The problem, two numbers. 2) Your solution in one sentence: an adaptive coach plus a camera spotter, free to run. 3) How it works: Flutter app, Kotlin MediaPipe on the phone, Python rules and a small ML model on a free server, Firestore for data. 4) What you proved: 95.7% posture-detection accuracy over 300 controlled reps (squat 91.3%, push-up 100%), a UAT with 20 participants (SUS 85.1, 95% overall satisfaction), an adaptive pipeline verified on device, and systematic testing that found and fixed defects. 5) What you did not prove and what you would do next. End with the one thing you are proudest of (the self-correction of the squat threshold from the literature is a strong candidate).

Demo script (8 to 10 minutes) and backups

#	Step	Talking point	Backup
0	2 minutes before: open `https://rakan-backend.onrender.com/`	Wake the free-tier server (cold start 52 s).	Say 'free tier' and show the plan already generated.
1	Home with an existing account (not onboarding live)	Plan, week indicator, hero card.	Screenshot deck.
2	Workout tab, then start a guided session	Equipment-matched plan; RPE capture.	Screen recording.
3	Posture detection: bodyweight squat, side-on, phone at hip height, 2 to 3 m	Skeleton, rep count, one deliberately shallow rep for red feedback, voice cue.	Short phone video of the same.
4	Finish session, show summary and PR	Atomic save, `/adapt-plan` result.	-
5	Plan changes dialog on the demo.deload account	Tier reasons with citations; -43% deload.	Screenshot.
6	Coach tab: stats, plateau notice (account B seeded history), recovery map	Pure-function plateau; MRV heat map.	Screenshots.
7	Log a knee injury; show the plan change	Injury-aware regeneration.	Skip if the server is slow.

Practical rules

• The emulator cannot run pose detection. **Demo posture only on the S20 FE**; charge it, plug in a cable, close other apps, disable auto-rotate surprises.

• Do not change the phone clock on stage (it breaks sign-in and the timing signal).

• Keep every number in Appendix A on one page you can glance at.

Typical 60-minute shape (confirm with your supervisor)

Minutes	Likely content
0 to 10	Your overview and demo, or the panel's opening questions.
10 to 35	Design and implementation questions (architecture, posture, adaptive engine).
35 to 50	Evaluation and testing, limitations, ethics.
50 to 60	Reflection, future work, closing questions.

Habits for answering: pause for two seconds; answer in one sentence first, then explain; say "I do not know, but this is how I would find out" rather than guessing; point at your own evidence ("my study shows", "my test covers"); never defend a mistake, explain how you found and fixed it.

Part C. Question bank (46 questions with answer outlines)

Use for the mock viva on day 19 and for daily drills afterwards. Cover the answer column, answer aloud in 30 to 60 seconds, then compare. Answers are outlines in plain words; say them in yours.

Idea

#	Question	Answer outline	Module
1	Why this problem? What is the gap?	Inactivity and dropout are large (31.3% inactive; about 50% quit in 6 months). Poor form causes injury. Most apps give a static plan or only video guidance. Rakan adapts the plan to logged effort and checks form on-device for free.	M0; proposal
2	Why did the posture target fall from 90% to 75%?	90% was set before single-camera 2D limits were known (Dill et al., 2023). 75% is a stated, citable minimum; you exceeded it in a controlled study. Be clear it was not lowered after seeing results.	M0
3	What is novel?	Combining a cited-threshold, rule-based posture engine with a propose/commit adaptive planner in one free, on-device app; plus a documented decision log.	M0, M12

Architecture

#	Question	Answer outline	Module
4	Why a backend at all?	Python for scikit-learn and rule code; plan writes with the Admin SDK. Cost: cold start and an extra moving part. Pure on-device port is future work.	M1
5	Why Firestore, not PostgreSQL?	No server to run, offline cache, rules, free tier. Cost: no joins, query-shaped modelling.	M1, M4
6	What if the server is asleep?	Plan generation shows a 90 s loading state; `/adapt-plan` failure is swallowed so the session still saves; measured cold start 52.37 s.	M1, M6
7	What fails first at 10,000 users?	Free-tier backend, unauthenticated endpoints, full-collection reads, Firestore cost. Say what you would change.	M12

Data and security

#	Question	Answer outline	Module
8	How is user A stopped from reading user B?	Owner-only rules under `users/{uid}/**`, verified email required, followers read only a weight-free `activityFeed`, follow docs validated by id and status. Proven by 25 scripted REST checks.	M3
9	Is the backend secure?	No, be direct: input ranges are validated, but the uid is trusted. ST-14 showed a plan written for another uid with no token. Fix: verify the ID token server-side.	M3
10	Why store a copy of each session as a feed item?	Privacy by structure: followers never get access to raw logs; weights are omitted.	M3, M4

Planner

#	Question	Answer outline	Module
11	Is the planner AI?	The planner is rule-based by design (explainable, testable, no training data). The AI parts are the learned fatigue model and the pose pipeline. State this plainly.	M5
12	How does the goal change the plan?	Only reps and rest via `GOAL_PRESCRIPTION`; exercise selection ignores goal. Say it before they find it.	M5
13	How do you know the generator is right?	Unit tests (split, session length, goal, locked days, injuries); Phase 28 review found four defects; device tests found two more.	M5

Posture

#	Question	Answer outline	Module
14	Why rule-based, not a trained classifier?	Actionable per-joint feedback, no labelled-fault dataset for your variants, thresholds traceable to papers, real-time on a phone. Evaluated with standard classification metrics.	M7
15	Why 2D and a side-on camera?	One phone camera is what users have; side-on keeps sagittal movement in the image plane. Cost: depth ambiguity, occlusion of the far limb.	M7
16	Where does 80 degrees come from?	Parallel squat inner knee about 60 to 70 degrees (Rojas-Jaramillo et al., 2024) plus the 10 degree tolerance.	M7
17	What is the 10 degree tolerance?	A named constant for 2D MediaPipe error (Dill et al., 2023). 20 degrees near full extension. One place to change, one value to test.	M7
18	Why separate counting and correctness thresholds?	So bad reps are counted and flagged; without it the confusion matrix has no false negatives by construction (Decision #62).	M7
19	Define TP, FP, FN, TN in your study.	Positive = incorrect form. TP = bad rep flagged. FP = good rep flagged as bad. FN = bad rep passed. TN = good rep passed.	M7
20	Why is push-up 100%?	Exaggerated faults separated the classes well; subtle faults not tested; real-world accuracy likely lower. Say it first.	M7
21	What are the 9 squat false positives?	Good reps the app flagged, mostly near thresholds (depth near 80, lean near 57) and tracking glitches. Pull the exact reps from the workbook before the viva.	M7
22	What about the four edited squat verdicts and the flips?	Four predicted-verdict cells in one set were edited by hand after the session. Every reported figure recomputes the verdict from the logged angle, so the edits do not change 91.3%. Each flip was based on what you saw. Sensitivity: 88 to 94%.	M7
23	How general is the result?	5 testers, 2 exercises, 1 phone, controlled conditions. It supports the claim for this setting, not for all body types or lighting.	M7, M12
24	Why no deadlift result?	No spine landmarks to judge rounding; analyser checks hinge and lockout only; data not collected. Limitation, not hidden.	M7
25	Is video stored or sent anywhere?	No. Frames are processed on the phone; only landmarks cross the channel to Dart.	M1, M7

Machine learning

#	Question	Answer outline	Module
26	What does R-squared 0.83 mean here?	The model reproduces a hand-written heuristic. It is not evidence of real-world prediction.	M8
27	Why synthetic data? Why not the Kaggle dataset?	Prepare your real reason. Likely: the dataset fields do not match what the app measures, and no labelled app data existed. Do not guess in the viva.	M8
28	Why linear regression?	Interpretable, six features, project rule of fundamentals first. A more complex model would only fit the same formula.	M8
29	Why thresholds 0.4 and 0.7 and the percentages?	Cited: Bell et al. (2025) for -17.5%; ACSM (2026) and NASM for +10%; maintain between. Single source of truth in `_apply_adaptation_rules`.	M8
30	Why is the experience coefficient negative?	Collinearity in the synthetic generator: experienced users have higher base RPE while the penalty favours beginners.	M8

Adaptive engine

#	Question	Answer outline	Module
31	Why weekly commits?	Avoid lurching on one bad session; atomic, reviewable change; summary with reasons.	M9
32	How are conflicting signals resolved?	Fixed tier order: scheduled deload, return from break, forced deload, weekly priority, session priority. Safety first.	M9
33	Where does 28 days come from?	Short-term detraining boundary of about 4 weeks (Mujika and Padilla, 2000; Encarnacao et al., 2022); tested at 28 and 29 days.	M9
34	Why is plateau not a signal?	It observes and never prescribes (Decision #57). Mixing them would overstate what the system does.	M9
35	Why does plateau need five sessions?	Four transitions need five values (Decision #59).	M9
36	What if a user changes the phone clock?	Return-from-break trusts the client clock (Decision #56), documented; server timing validation is future work.	M9
37	How do you know adaptation helps users?	Not yet measured. Logic verified by unit and device tests. Propose a 4-week pilot.	M9, M12

Testing

#	Question	Answer outline	Module
38	How do you know it works?	Layered evidence: unit tests, scripted rules tests, emulator system tests with named IDs, physical-device posture tests, accuracy study.	M11
39	A bug that unit tests could not catch?	Experience always sent as beginner (wrong key on client) or completion always 1.0; the backend was correct, the client built the wrong input.	M8, M11
40	Why split pure logic from Firestore code?	Testability: boundary tests run in milliseconds without a database.	M9, M11
41	Why can't you test pose on the emulator?	x86 emulator cannot load the MediaPipe native library; physical S20 FE only.	M7, M11
42	Did users actually like it? How did you measure satisfaction?	UAT with 20 consenting participants: SUS mean 85.1 (SD 6.6), above the commonly cited average of 68. 95% agreed they were satisfied overall. Weakest item: posture feedback (85% agree); comments ask for steadier detection and easier phone placement.	M11

Reflection

#	Question	Answer outline	Module
43	What was your biggest mistake?	Pick a true one with evidence: the half-squat threshold, or goals having no effect on prescription. Say how you found it (review, literature) and what changed.	M7, M5
44	What would you do with six more months?	Real user study, server-side auth, larger blinded posture study, single exercise library, iOS pose.	M12
45	What are the ethical issues?	Not medical advice, injury logging, data minimisation, anonymised testers and consent, no video leaving the phone.	M12
46	What are you most proud of?	Choose one and give evidence, e.g. the decision log with 64 entries or correcting thresholds from the literature.	M12

Appendix A. One-page numbers card (verify before the viva)

Topic	Numbers
Project	Started 9 Apr 2026; 124 commits to 972060c; Flutter 3.41.6; Python 3.12; Firebase project `rakan-10680`; test phone Samsung S20 FE.
Library	Flutter 626 exercises (126 hand-written + 500 from dataset); backend pool 326 (126 + 200); 31 exercises with pose detection.
Plan generator	Exercises per session: beginner 3 to 6, intermediate 3 to 7, advanced 3 to 8. 10 min warm-up/cool-down. 45 s per working set. Max 5 sets.
Fatigue model	6 features; 1,000 synthetic rows, seed 42; 80/20 split; MAE 0.0404; R-squared 0.829. Levels: > 0.7 high (-17.5%), 0.4 to 0.7 medium (0%), < 0.4 low (+10%).
Adaptive tiers	Week % 4 == 0: -42.5%. Return from break: <= 28 d 0%, > 28 d -17.5%. Recovery > 0.8: forced deload min(session, -17.5%). Recovery < 0.5: max(session, trend). Else session. Trend: > +5% increasing, < -10% decreasing.
Recovery	7-day window; primary set 1.0, secondary 0.5; MRV: back, chest, legs, glutes 20; shoulders, arms 16; core 12; rest days 2 if score < 0.5 else 3.
Plan scaling clamps	sets 1 to 6; reps 3 to 20; weight never modified.
Missed days	Scan 30 days back; reschedule needs >= 48 h after last trained; auto-expire after 7 days.
Plateau	N = 4 transitions, 5 sessions; threshold 2% inclusive; passive notice.
Posture rule	Threshold = reference + 10 degrees (20 near extension).
Squat	Counted <= 110; depth <= 80; standing >= 160; trunk lean <= 57.
Push-up	Counted <= 120; depth <= 100; body line >= 160.
Lunge / RDL / press / curl	Lunge <= 100 and lean <= 30; RDL knee >= 155 and depth <= 110; press lockout >= 160 and asymmetry <= 20; curl <= 50 and swing <= 30.
Accuracy study	5 testers x 3 sets x 10 reps x 2 exercises = 300 reps. Push-up 100% (97.5 to 100); squat 91.3% (85.7 to 94.9); overall 95.7% (92.7 to 97.5). Squat TP 69, TN 68, FP 9, FN 4. Squat sensitivity 88% (no flips) to 94% (as logged).
Security / ops	Login: 5 free failures then 30 s doubling, cap 15 min. Cold start 52.37 s. Comments 1 to 500 characters. Feed omits weights.
Tests	105 Dart tests passed on 18 Oct; rules checks 25/25; ST-14 shows backend unauthenticated. **Re-run for fresh numbers.**
UAT	22 to 23 Oct; 20 participants (P01 to P20), consent recorded. SUS 85.1 (SD 6.6, range 72.5 to 95). Agree or strongly agree: onboarding 100%, plan suited 90%, logging 100%, posture feedback 85%, stats 100%, plan-change explanations 90%, design 95%, overall satisfaction 95% (target 75%).

Appendix B. Sources and what each one supports

Full Harvard references live in your dissertation and in `docs/POSTURE_THRESHOLDS.md`. Know the claim each source supports; examiners ask "what does that paper actually say?".

Source	Used for
Strain et al. (2024), Lancet Global Health	31.3% of adults insufficiently active in 2022 (problem statement).
Linke, Gallo and Norman (2011)	About 50% dropout within six months (problem statement).
Chae et al. (2023), Interactive Journal of Medical Research	AI exercise coaching app improved posture correction in an RCT.
Dill et al. (2023), BMT poster	MediaPipe 3D pose accuracy best for standing exercises, degrades as joints near straight; basis of the tolerance.
Rojas-Jaramillo et al. (2024), Frontiers in Sports and Active Living	Squat depth categories (parallel about 60 to 70 degrees); deep squats safe for healthy knees (13 of 15 studies).
Graber et al. (2023), Physical Therapy in Sport	Trunk lean at parallel depth 37.8 +/- 9.7 degrees; basis of the 57 degree limit.
Farrokhi et al. (2008), JOSPT	Lunge kinematics and the trunk-forward variant; lunge thresholds.
Chua et al. (2024), arXiv	Fatigue-induced compensation in bicep curls; upper-arm swing limit.
Escamilla (2001), Sinclair et al. (2016)	Squat biomechanics; front vs back squat posture.
Topend Sports (n.d.), Physiopedia (n.d.)	FitnessGram push-up 90 degree standard; Romanian deadlift cues (non-peer-reviewed; say so).
Zourdos et al. (2016)	RPE-based load regulation; the fallback `avg_rpe / 10`.
Bell et al. (2025)	Reactive deload of -17.5%.
ACSM position stand (2026), NASM	5 to 10% progressive overload; RPE 6 to 7 maintain.
Israetel et al. (volume landmarks)	Weekly MRV values by muscle group.
Gjestvang et al. (2023)	Missed sessions and dropout; behavioural framing of the skip trigger.
Mujika and Padilla (2000); Encarnacao et al. (2022)	Detraining: adaptations retained for about 4 weeks.
Brooke (1996); Sauro (2011); Bangor, Kortum and Miller (2009)	SUS questionnaire and scoring; 68 as the commonly cited average; adjective scale (mid-80s is about "excellent"). Use whichever SUS references your report already cites.

Rule for the viva

• For every citation, be able to say: what the study did, what number you took from it, and what you added (tolerance, estimate). Where you added an engineering estimate, say that it is an estimate.

Appendix C. File reading map (largest hand-written files)

About 30k lines of hand-written code. Data files (`exercise_data_extended.dart` 9.9k lines, `exercises_source.json` 146k lines, `exercises.py`, `exercises_extended.py`) are content, not logic: know what they are and skip reading them.

Area	File	Lines	Study depth
Posture	`features/workout/services/angle_calculator.dart`	1,452	Deep
Posture	`android/.../PoseDetectorHandler.kt`	219	Deep
Posture	`features/workout/screens/pose_detection_screen.dart`	847	Medium
Posture	`voice_coach.dart`, `form_check_hud.dart`, `pose_service.dart`	198 / 185 / 64	Light
Adaptive	`features/workout/services/adapt_service.dart`	693	Deep
Adaptive	`backend/services/adaptation_engine.py`	203	Deep
Adaptive	`features/workout/services/weekly_summary_service.dart`	419	Deep
Adaptive	`backend/routers/adapt_router.py`, `adaptation_engine_router.py`	132 / 73	Deep
ML	`backend/ml/generate_training_data.py`, `train_model.py`	56 / 69	Deep
Planner	`backend/services/plan_generator.py`	565	Deep
Planner	`backend/routers/plan_router.py`	198	Medium
Logging	`features/workout/services/workout_log_service.dart`	549	Deep
Logging	`workout_active_screen.dart`, `auto_log_screen.dart`, `workout_complete_screen.dart`	2,226 / 2,092 / 885	Medium (skim UI; read the state logic)
Schedule	`schedule_matcher.dart`, `workout_plan_service.dart`	97 / 480	Medium
Auth	`auth_service.dart`, `login_throttle.dart`, `auth_navigation_service.dart`, `firestore.rules`	182 / 52 / 40 / 129	Deep
Onboarding	`onboarding_data.dart`, `user_profile_service.dart`, steps 1 to 8	319 / 133 / ~3,200	Medium
Coach	`training_stats.dart`, `injury_service.dart`, `plan_reset_flow.dart`	405 / 202 / 213	Medium
Coach UI	`coach_screen.dart`, `stats_report_tab.dart`, `settings_screen.dart`, `home_screen.dart`	1,803 / 1,327 / 1,023 / 1,558	Light (UI)
Social	`follow_service.dart`, `public_profile_service.dart`, `activity_interaction_service.dart`	161 / 123 / 106	Light
Notifications	`notification_service.dart`	475	Light
Tests	`adapt_service_test.dart`, `posture_variants_test.dart`, backend tests	737 / 330 / 571	Read as specification
Study tools	`tools/posture_study/capture.py`, `analyse.py`	427 / 212	Medium
Docs	`HANDOVER.md`, `SYSTEM_OVERVIEW.md`, `docs/POSTURE_THRESHOLDS.md`	617 / 531 / 258	Read fully

Last advice

- Your strongest asset is that every claim is traceable: a decision number, a paper, a test, a measurement. In the viva, point at that trail.
- If you do not know, say what you would check and where. That reads as engineering maturity, not weakness.